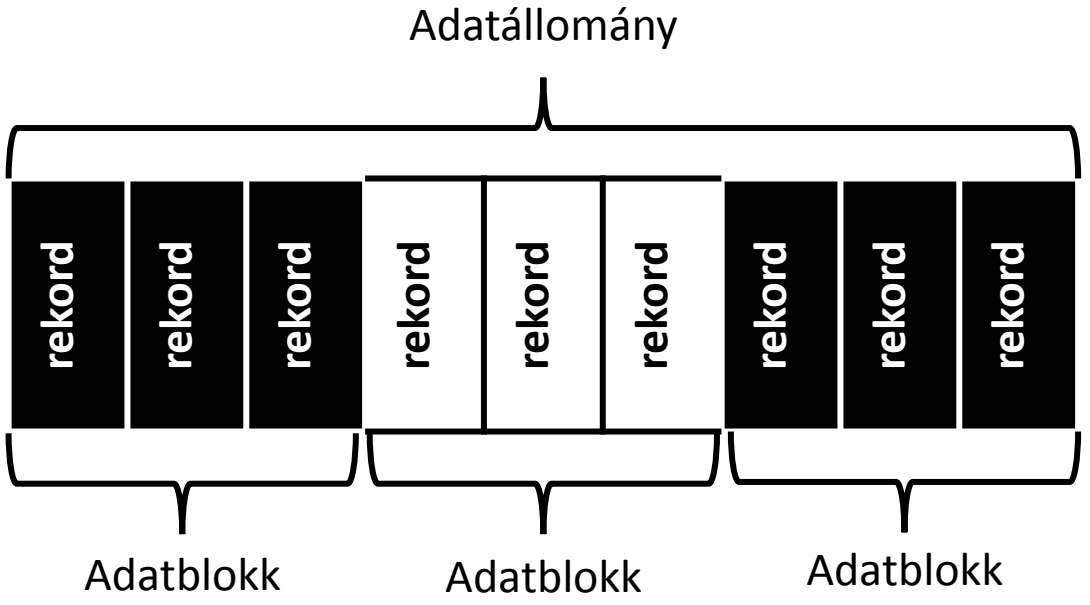
Fizikai adatszervezés

Dr. Gajdos Sándor

Dr. Erős Levente diái alapján, 2026

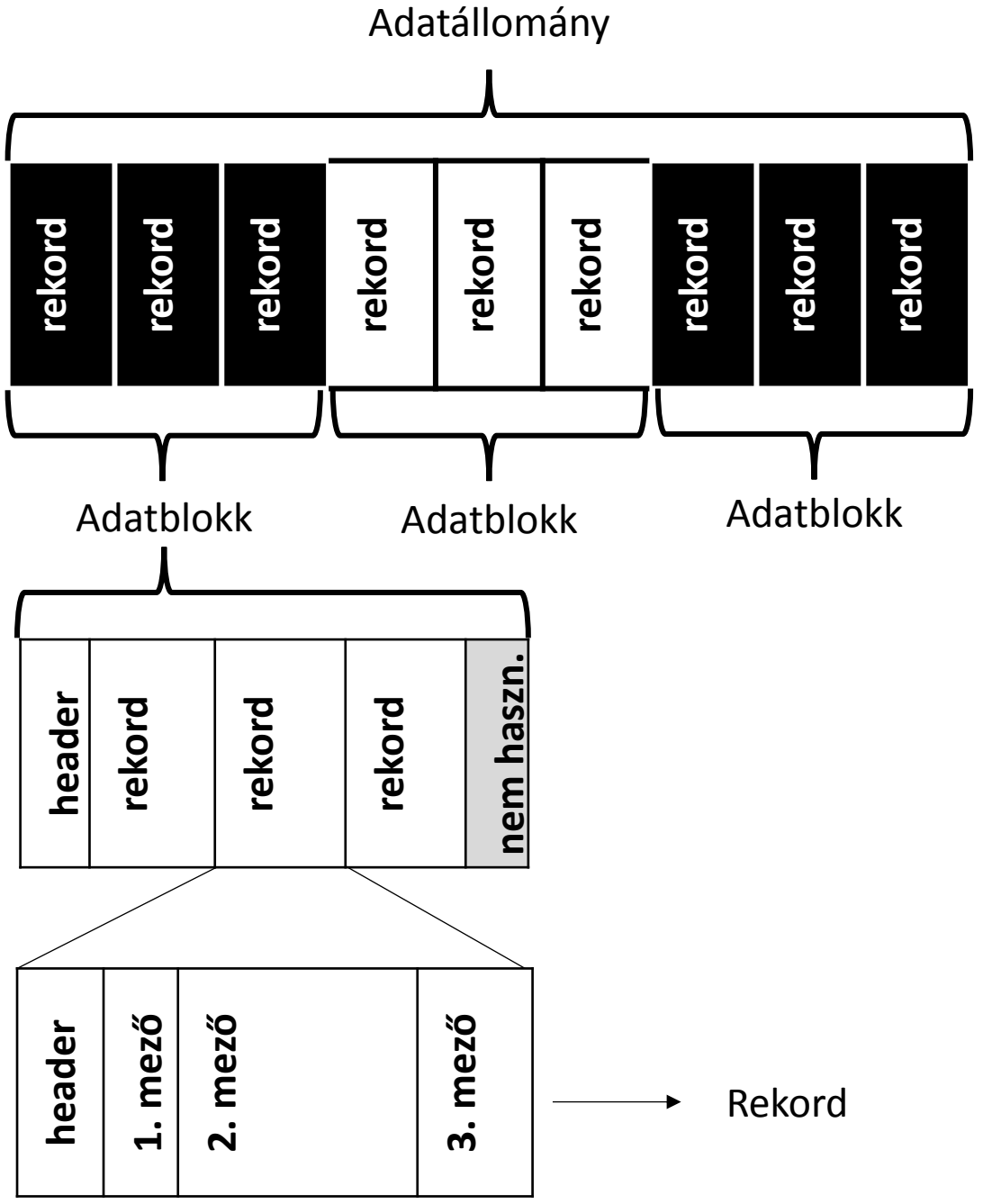
Alapvetések



- Hol tartunk?
  - Háromrétegű DBMS-modell
    - Felhasználói nézetek
    - Logikai (fogalmi, koncepcionális) adatbázis (ld. ER, rel. adatmodell)
    - **Fizikai adatbázis (aktuális témakör)**
- Alapvetések
  - Az adatokat (általában) egy **adatállományban** tároljuk a háttértáron → „állományszervezés” vs. „adatszervezés”
  - A fizikai eszköz: HDD, SSD („block device” vö. „character device”)
  - Az adatállomány egységes méretű **blokkokból** áll (tip.: néhány kbyte méretű)
  - Minden blokknak van **címe**, amelyet mutatók használnak
  - A mutatók lehetnek **fizikai** vagy **logikai** mutatók.
  - Az operációs rendszer az adatállományt **folytonos** blokksorozatként jeleníti meg a DBMS számára

Adatállomány

- Blokkszerkezet
  - Header/fejléc (státusz, mutatók,…)
  - Egységes méretű *rekordszerű* struktúrákat tartalmaz (n-es, tupel, rekord, objektum,…)
  - Nem használt terület
- Rekordszerkezet
  - header/fejléc (mutatók, törölt bit, használt bit,…)
  - Különböző méretű és típusú mezők
  - Rekordméret: egy típuson belül általában állandó, de lehet változó is
    - Ismétlődő mezőcsoport, változó hosszúságú mező
- Jelölések
  - b – blokkméret (egységes)
  - n_r – rekordszám
  - s_r – rekordméret, egy blokkban egyforma
  - $f_r = \left\lfloor \frac{b}{s_r} \right\rfloor$ – rekordszám blokkonként (blocking factor)
  - $b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$ – állomány blokkszáma

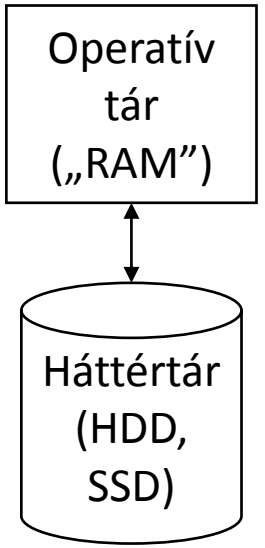


Tárolt adatok fajtái

- A „felhasználói adatbázisban” található adatok
- Segédadatok
  - Keresőstruktúrák
  - Adatokról tárolt adatok – metaadatok
    - Strukturális ~
    - Szemantikus ~
    - Üzemeltetési ~

Adathozzáférés a háttértáron

- Rekordhozzáférés
  - Teljes blokkot olvasunk a memóriába (10…200µsec, ill. 0,5…2msec)
  - Egy blokkmozgatás – egy blokkművelet
  - Alapértelmezetten a memóriába egy blokk fér (így számolunk)
- 4 művelet
  - **Keresés**, törlés, beszúrás, módosítás
    - A keresés hatékonyságától függ a többi hatékonysága
- Def.: Keresési kulcs: keresett attribútum – nem feltétlenül biztosít egyediséget



Alapvető állományszervezési módszerek

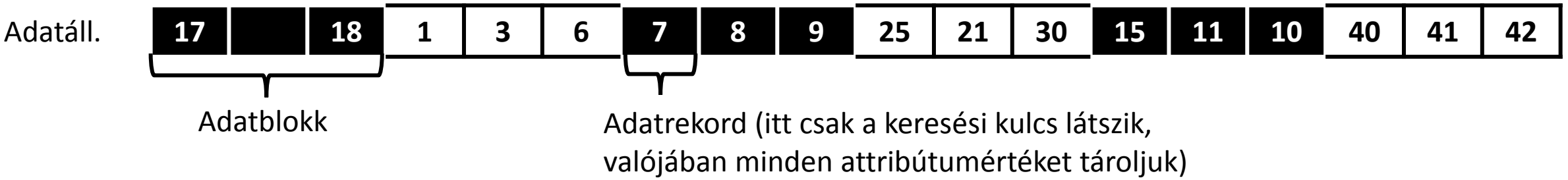
- Heap
- Hash
- Indexelt (bizonyos fajtái)

Az összehasonlítás alapja: blokkműveletek száma

- Nagyságrenddel több időt vesz el, mint egy blokknak a memóriában történő feldolgozása

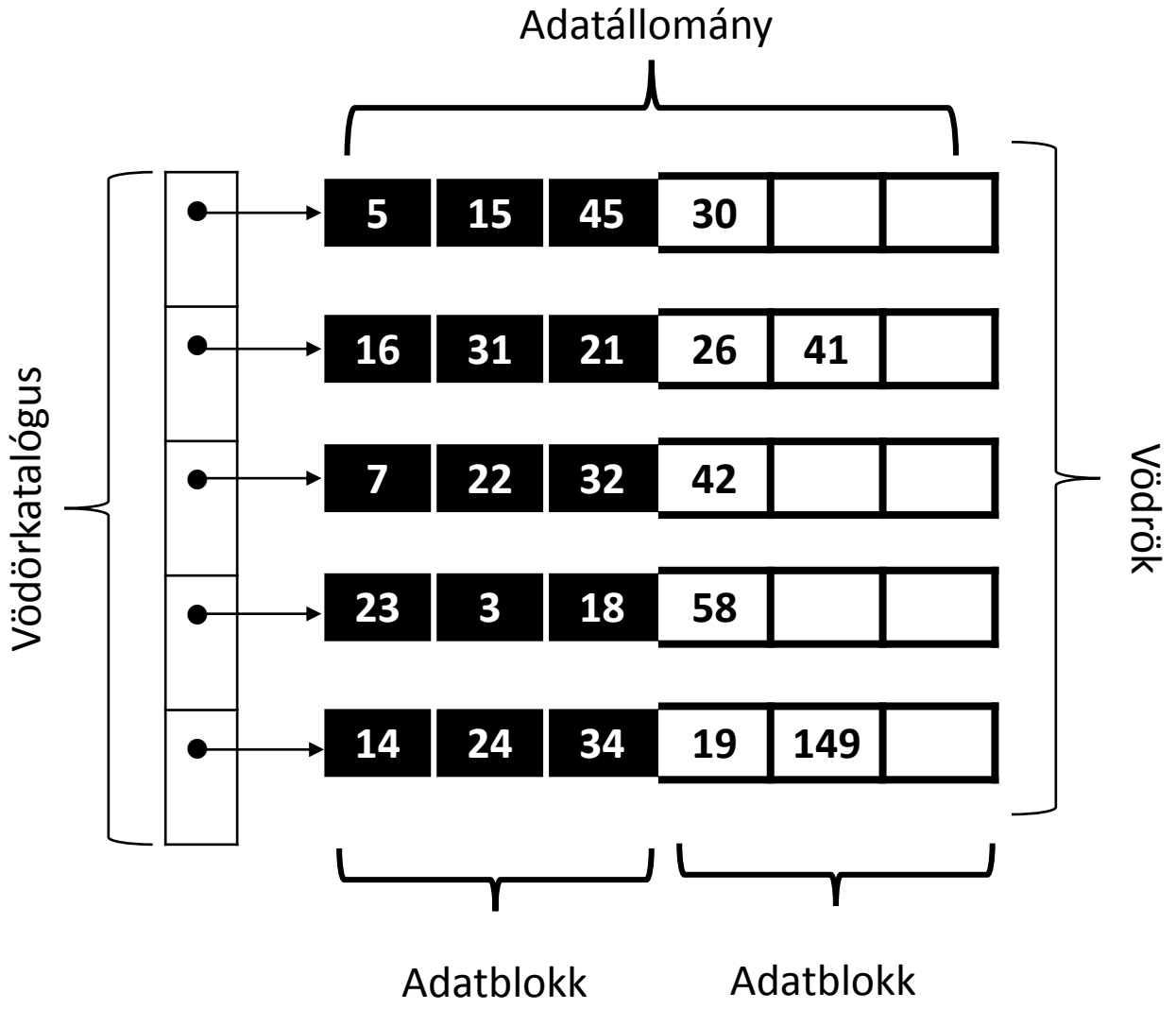
Heap

- Nincs keresési segédstruktúra
- Nem feltétlenül rendezettek a rekordok
- Keresés: lineáris, blokkok olvasása a memóriába egymás után
  - Ha kulcsattribútumot keresünk, megállunk, amikor megvan – átlagosan $(1+b_r)/2$ blokkművelet
  - Egyébként a fájl végéig keresünk – b_r blokkművelet
- Törlés: keresés, törölt bit beállítása, módosult blokk visszaírása
- Beszúrás: keresés (egyediség!), blokk módosítása, blokk visszaírása
- Módosítás: keresés, mező(k) módosítása, módosult blokk visszaírása



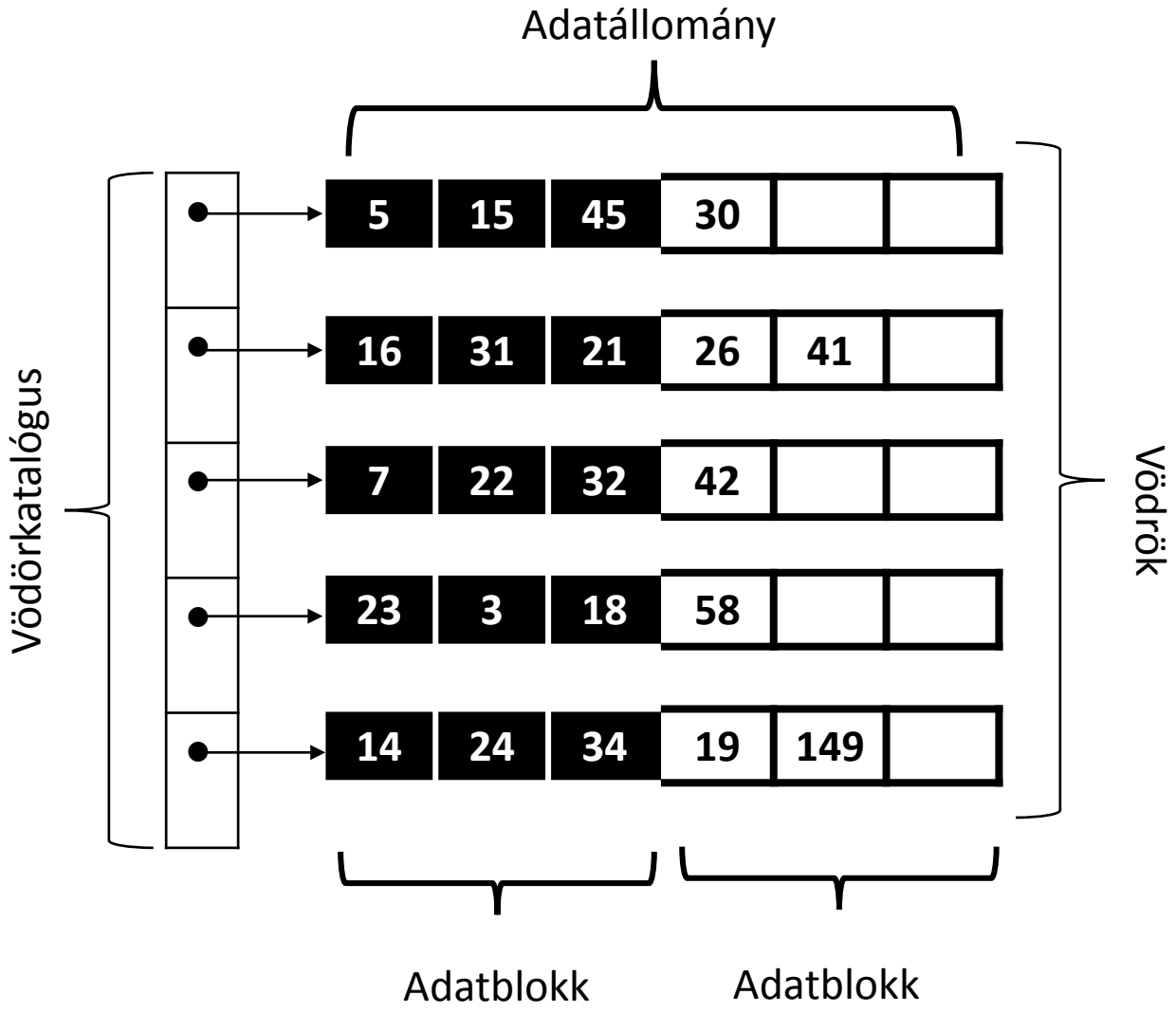
Vödrös hash I.

- Az adatállományt ú.n. vödrökre osztjuk
  - Blokkok listája
  - Vödrök száma: B
- Vödörkatalógus (hash-tábla)
  - **Mutatók** tömbje
    Általában befér a memóriába (ezt mindig feltételezzük)
- Hash függvény & keresés
  - h(K) → [0..B-1] (Pl: h(K)=(c*K) mod B)
    - hash_table[h(K)] mutatót követjük
    - Vödrön belül lineáris keresés („mini heap”)
  - A jó hash függvény egyenletesen osztja el a h(K) értékeit 0..B-1 között



Vödrös hash II.

- Keresés átlagos ideje: (1+N/B)/2
- Előny: alkalmas B mellett igen gyors lehet
- Hátrány: intervallumkeresést nem támogat



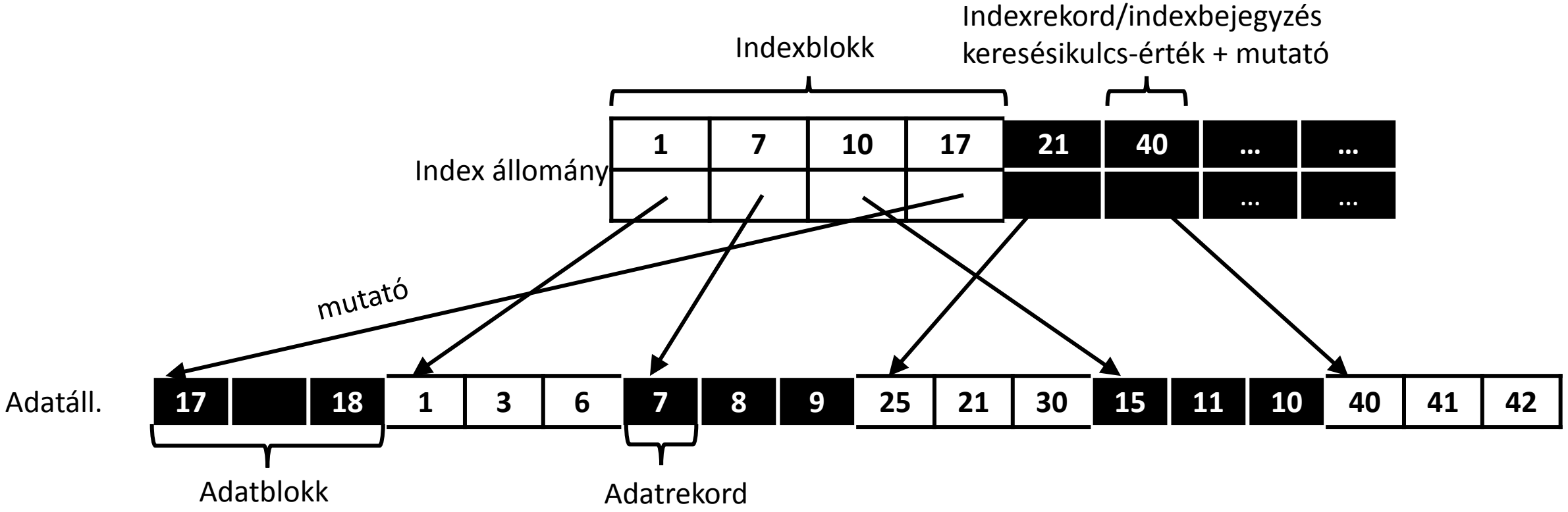
Indexelt állományszervezés

- Def.: Index (keresési) kulcsérték+mutató
- Segédstruktúra: indexrekordokból álló indexállomány
  - Az index általában jóval kisebb, mint az adatállománya
  - Az index mindig rendezett

  → benne a keresés jóval hatékonyabb lehet
- Indexelés fajtái:
  - Ritka index: # indexrekordok << # adatrekordok
    - Egyszintű („flat”, pl. ISAM)
    - Többszintű (pl. B*-fa)
  - Sűrű index: # indexrekordok = # adatrekordok
    - Önmagában nem használatos állományszervezésre

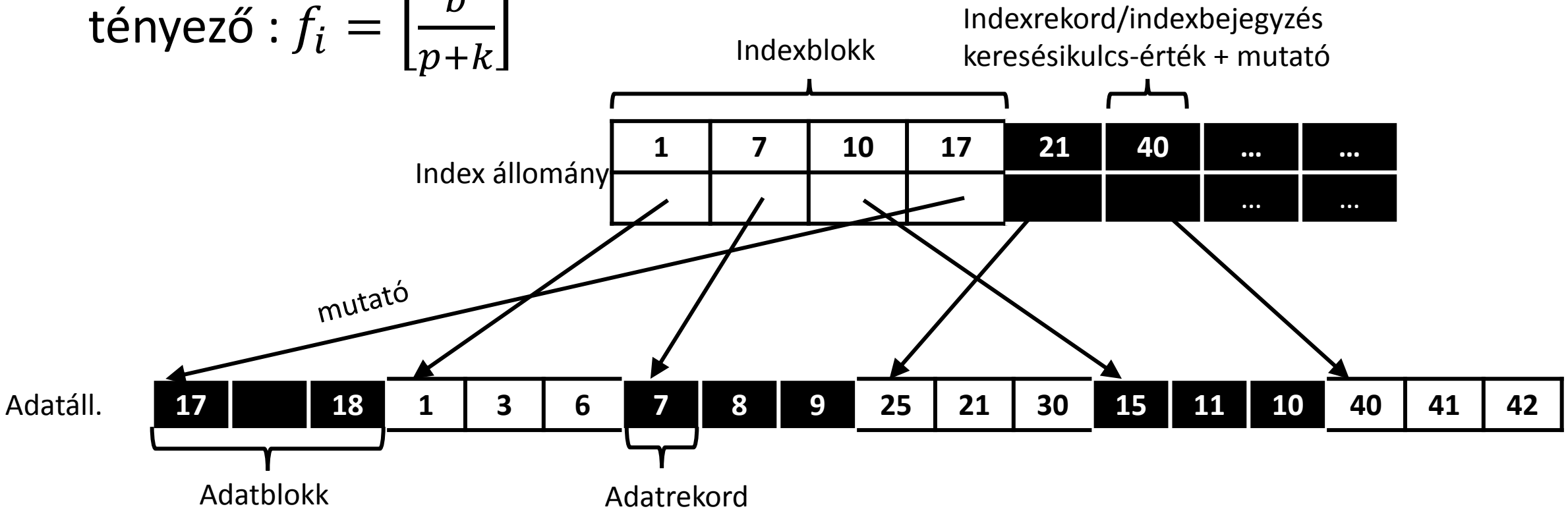
Ritka index példa: ISAM I.

- Az index mutatói adatblokkokat azonosítanak
- Minden adatblokkhoz tartozik 1 indexbejegyzés
- Az indexrekord kulcsértéke a hozzá tartozó adatblokkban lévő (pl.) legkisebb kulcsérték
- Adatállomány rendezett (…), szótár analógia



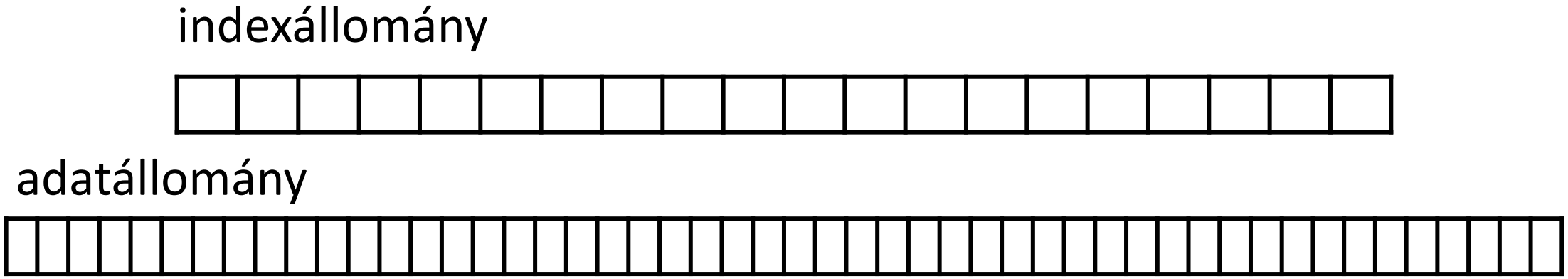
Ritka index példa: ISAM II.

blocking factor/elágazási tényező : $f_i = \left\lfloor \frac{b}{p+k} \right\rfloor$



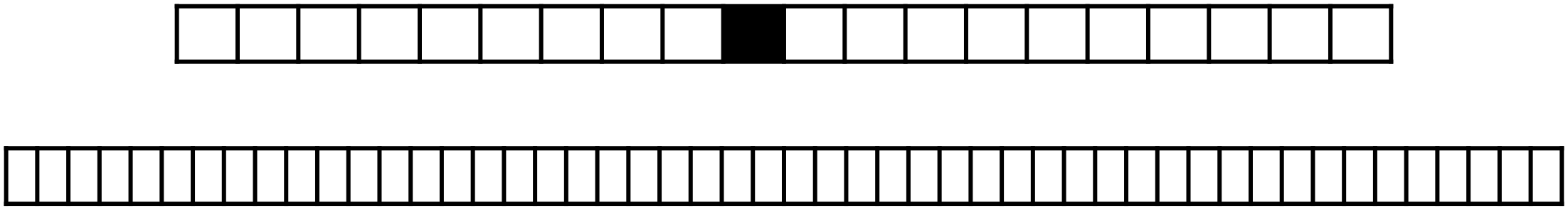
Ritka index, keresés

- Csak a blokkokat mutatjuk



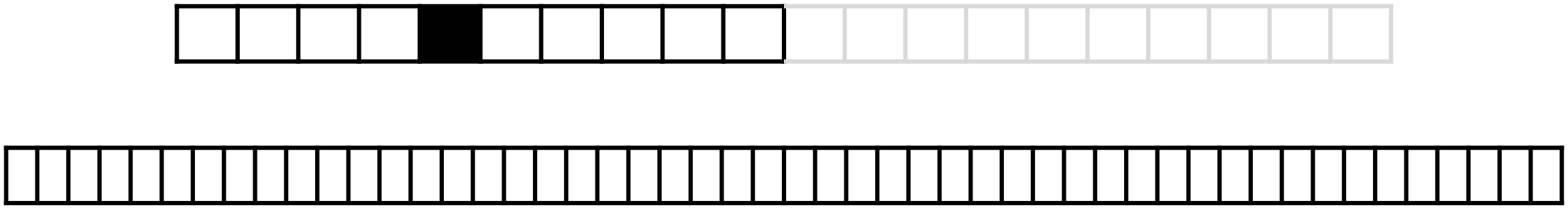
Ritka index, keresés

- Csak a blokkokat mutatjuk



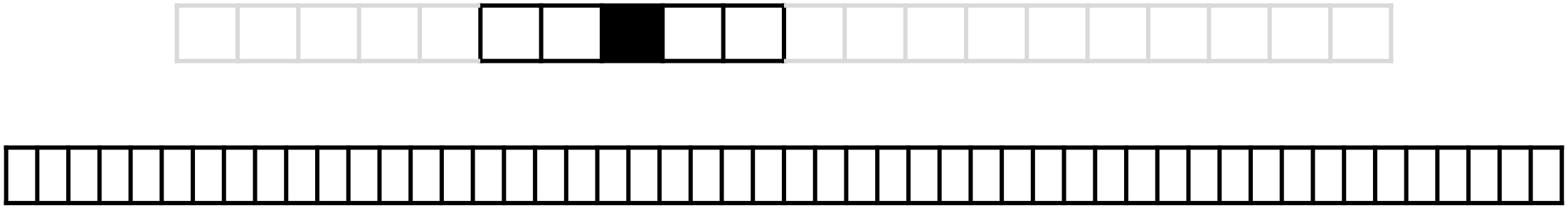
Ritka index, keresés

- Csak a blokkokat mutatjuk



Ritka index, keresés

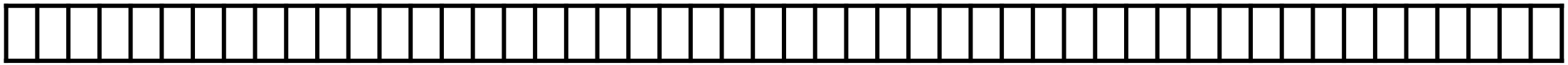
- Csak a blokkokat mutatjuk



Ritka index, keresés

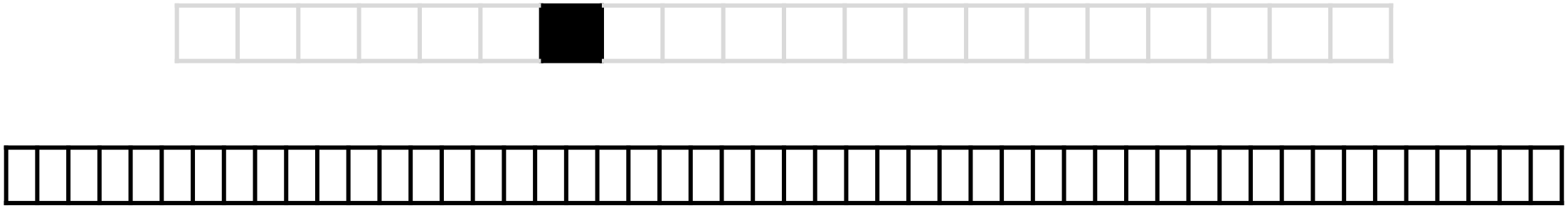
- Csak a blokkokat mutatjuk





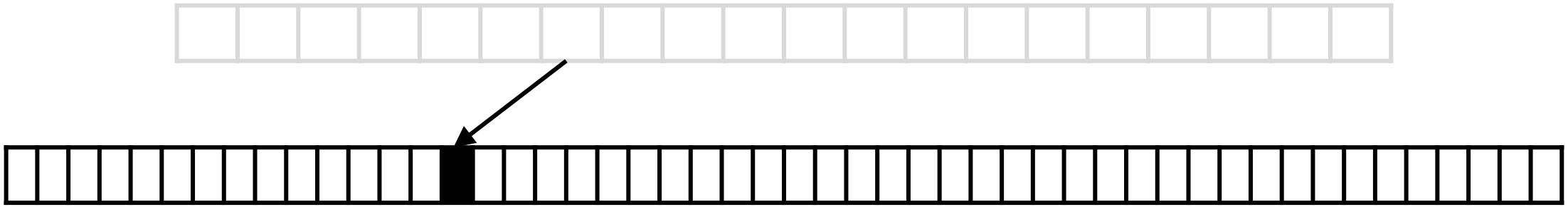
Ritka index, keresés

- Csak a blokkokat mutatjuk



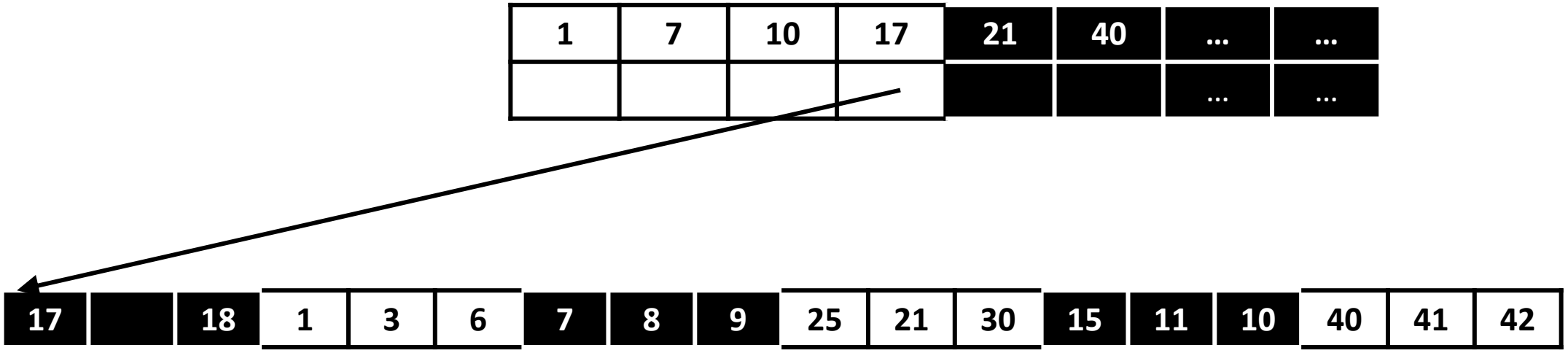
Ritka index, keresés

- Csak a blokkokat mutatjuk
- Bináris keresés + 1 blokkolvasás az adatállományból
- $\lceil \log_2(b_i + 1) \rceil + 1$ $\quad\quad$ b_i: az indexáll. blokkjainak száma



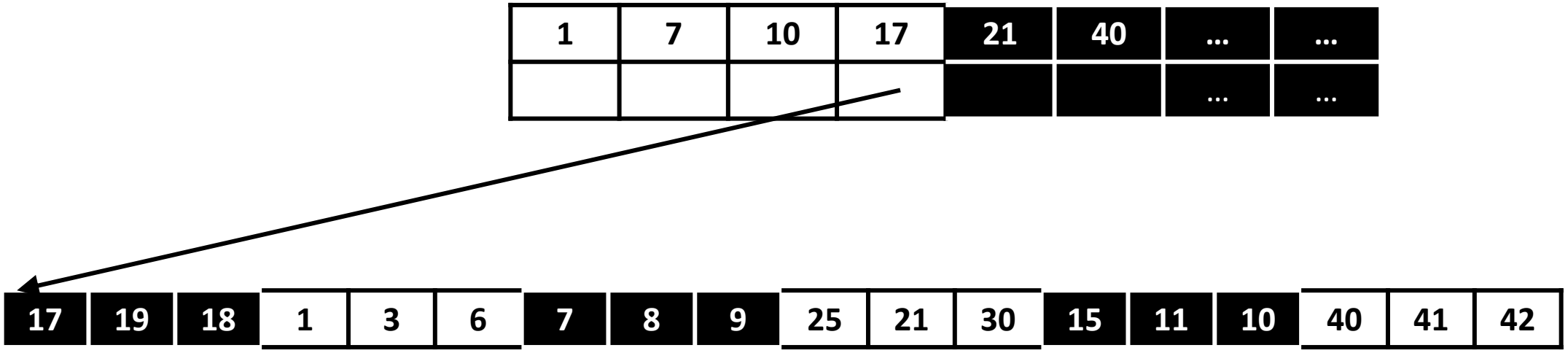
Ritka index, beszúrás

- Keresés+beszúrás a megfelelő blokkba
- 1. eset: 19 beszúrása



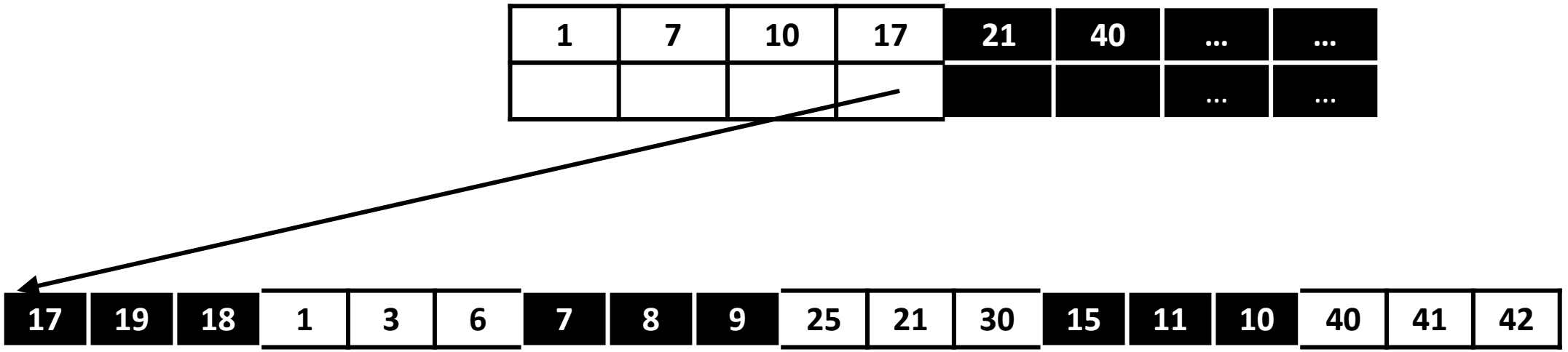
Ritka index, beszúrás

- Keresés+beszúrás a megfelelő blokkba
- 1. eset: 19 beszúrása



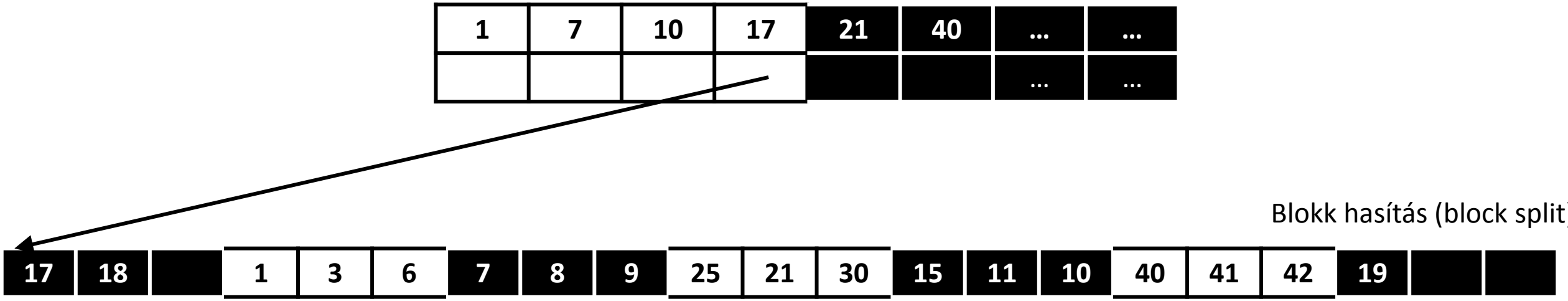
Ritka index, beszúrás

- Keresés+beszúrás a megfelelő blokkba
- 2. eset: 20 beszúrása, a célblokk ugyanaz, de nincs benne hely



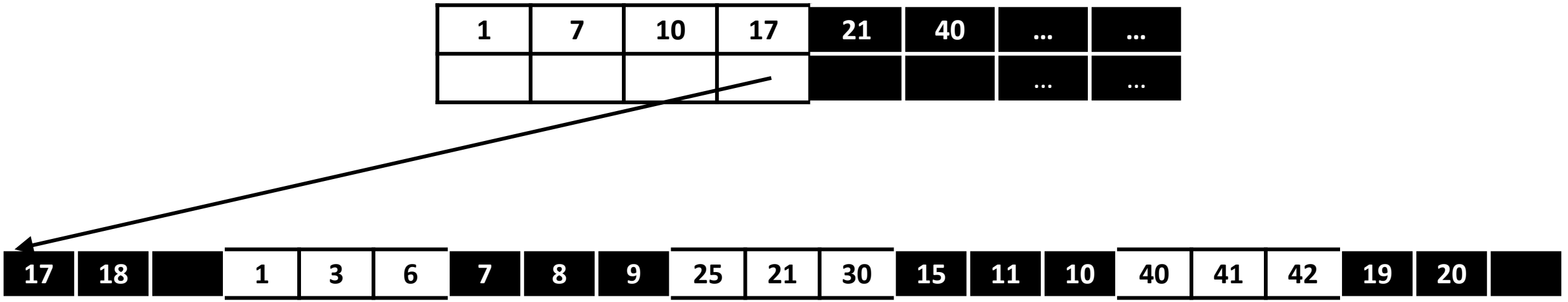
Ritka index, beszúrás

- Keresés+beszúrás a megfelelő blokkba
- 2. eset: 20 beszúrása, adatblokk hasítása, majd indexblokk hasítása, ha kell



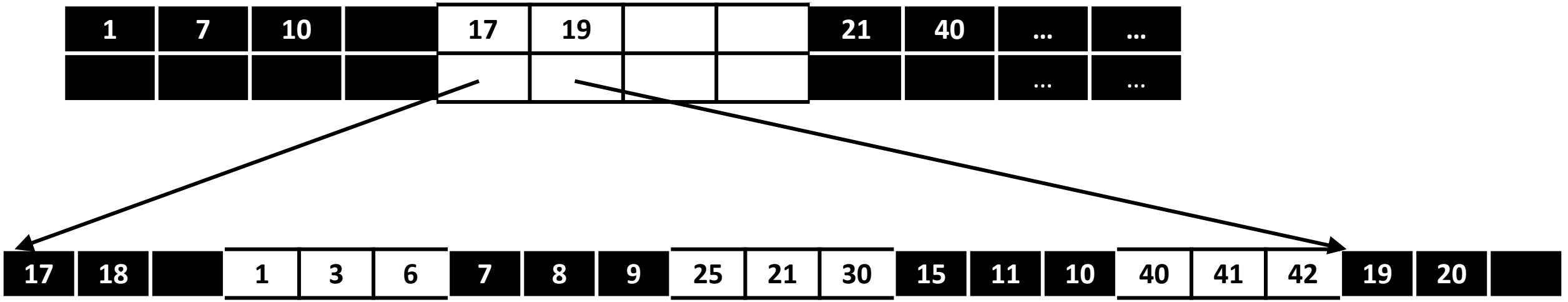
Ritka index, beszúrás

- Keresés+beszúrás a megfelelő blokkba
- 2. eset: 20 beszúrása, adatblokk hasítása, indexblokk hasítása, ha kell



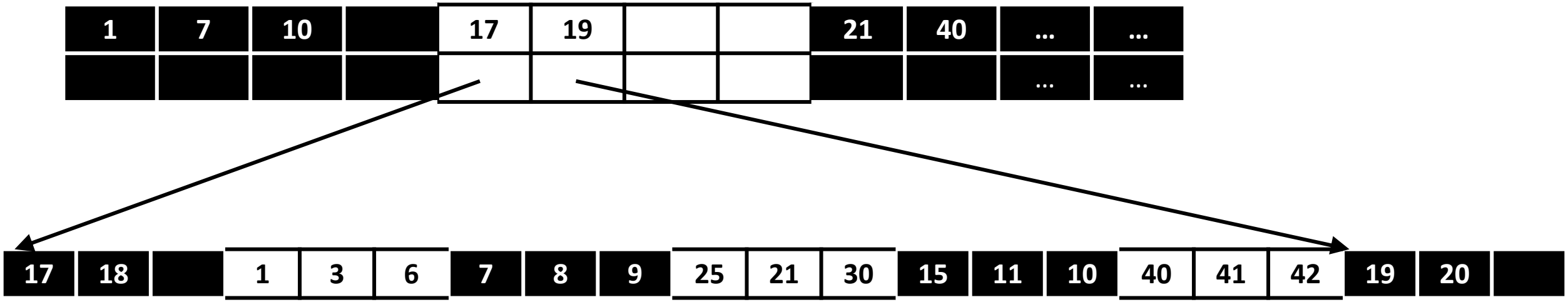
Ritka index, beszúrás

- Keresés+beillesztés a megfelelő blokkba
- 2. eset: 20 beszúrása, adatblokk hasítása, indexblokk hasítása, ha kell



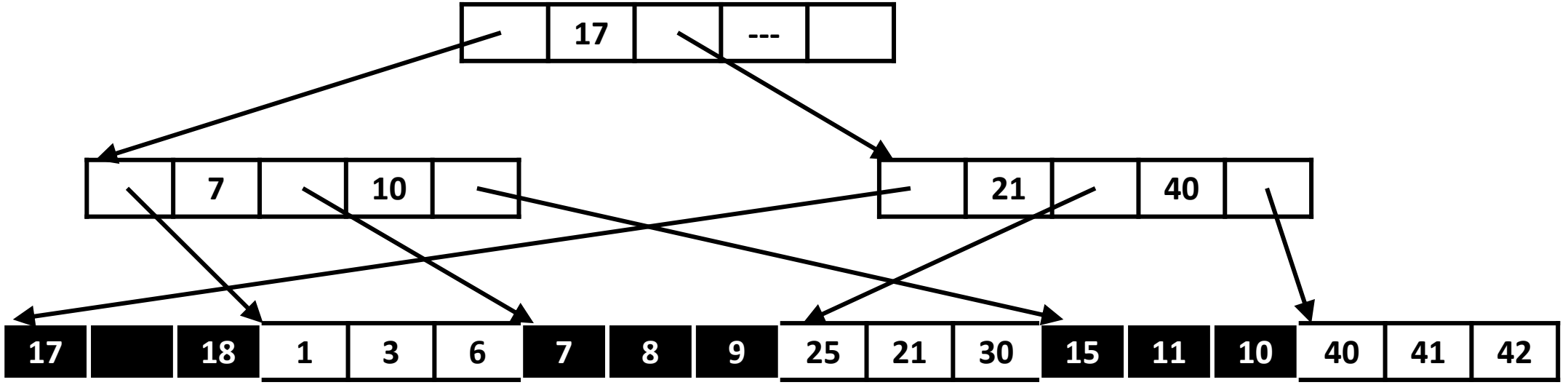
Ritka index: törlés, módosítás

- Törlés: block split ellentéte lehet szükséges
- Módosítás:
  - egyszerű, ha nem érint kulcsot
  - Törlés + beszúrás, ha kulcsértéket érint



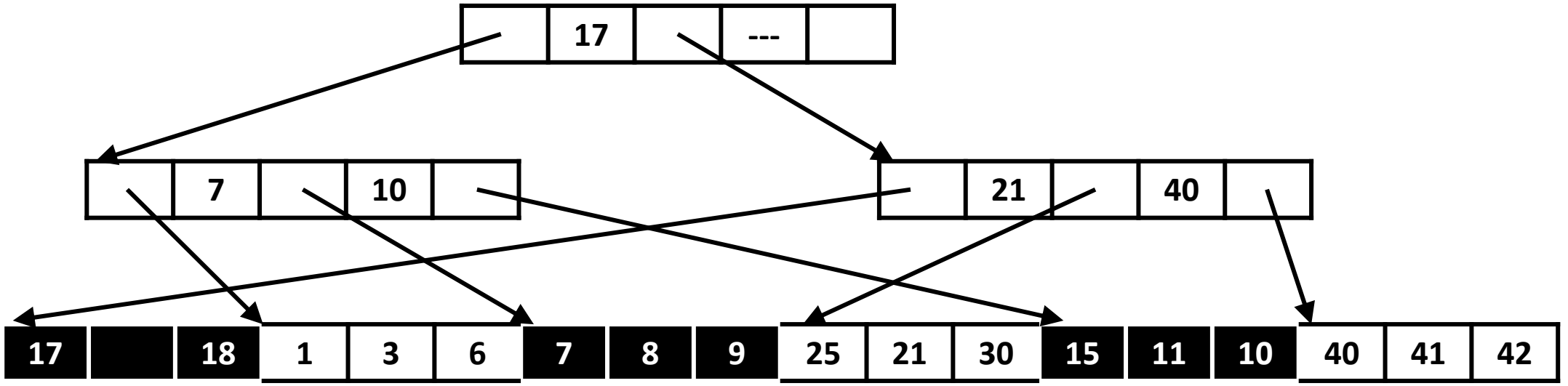
B*-fa

- Egyfajta többszintű ritka index
- Ritka index ritka indexe,…
  - A legfelső szinten 1 indexblokk – fa gyökere
- Indexblokkokban most kulcsértékek és mutatók egymás után, az első kulcsérték hiányzik
- Ha a rendezés növekvő, akkor
  - A K kulcstól balra lévő mutató egy olyan részfára mutat, amelyben minden érték < K
  - A K kulcstól jobbra lévő mutató egy olyan részfára mutat, amelyben minden érték >= K
- Intervallumkereséshez az alsó szintű indexblokkokat előre és hátrafelé is összeláncolják



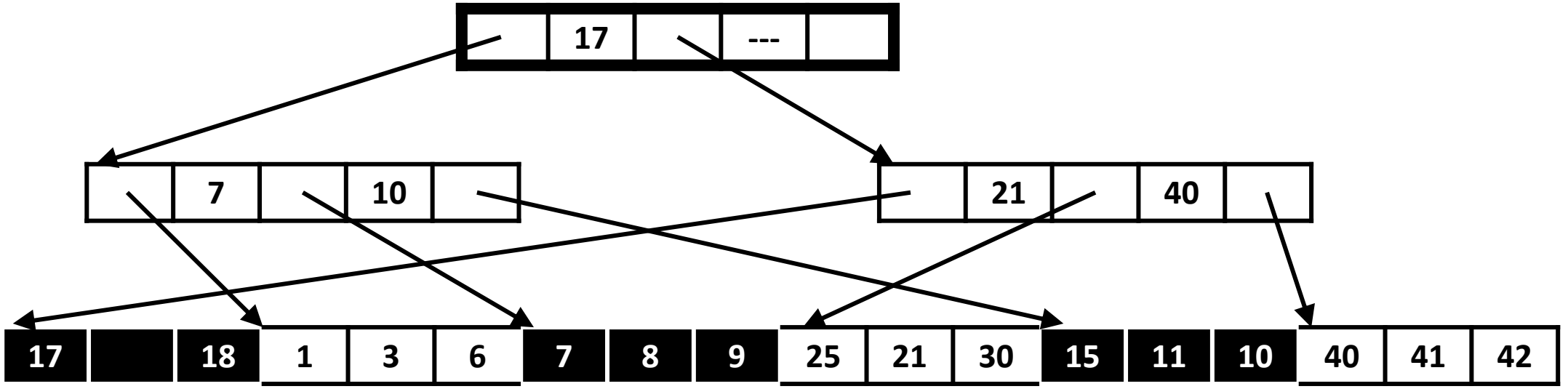
B*-fa (Miért éppen B?)

- A fa csomópontjai blokkok
- Kiegyensúlyozott („balanced”)
- Index blocking faktora = elágazási tényező = $f_i = \left\lfloor \frac{b+k}{p+k} \right\rfloor$
  - Indexblokkonkénti mutatók maximális száma
- HT_i – height of tree – fa magassága



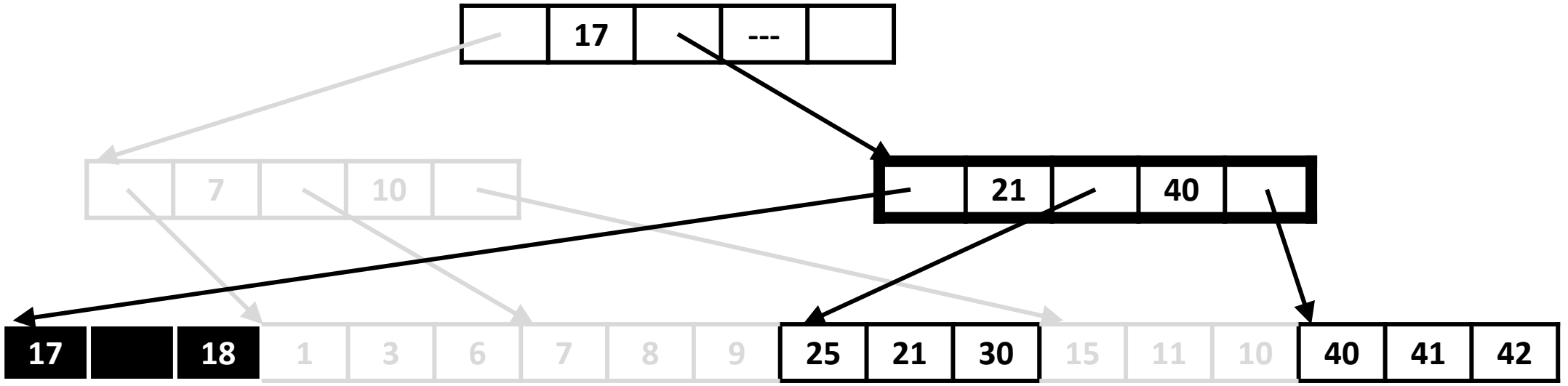
B*-fa, keresés

- 18 keresése



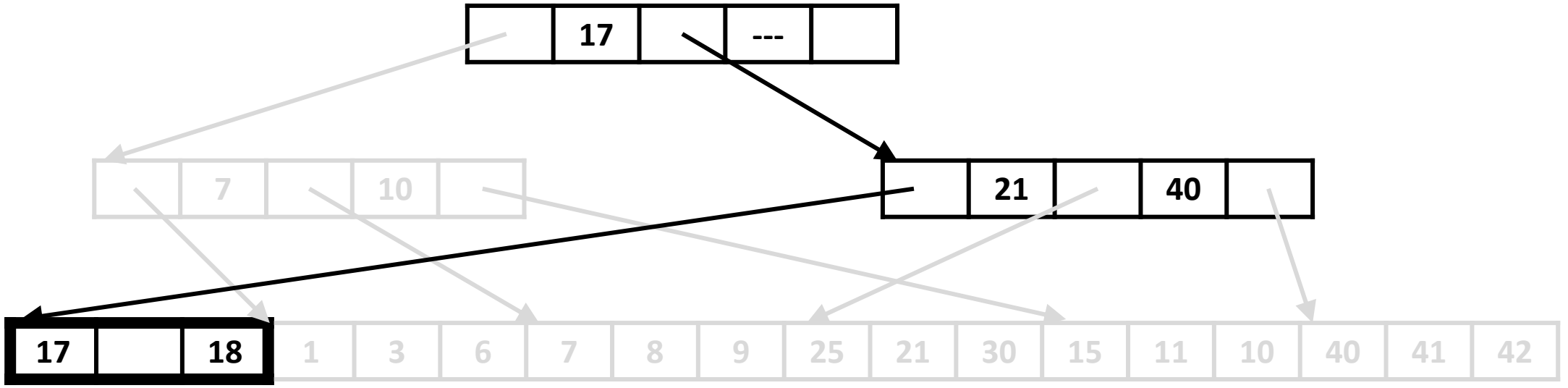
B*-fa, keresés

- 18 keresése



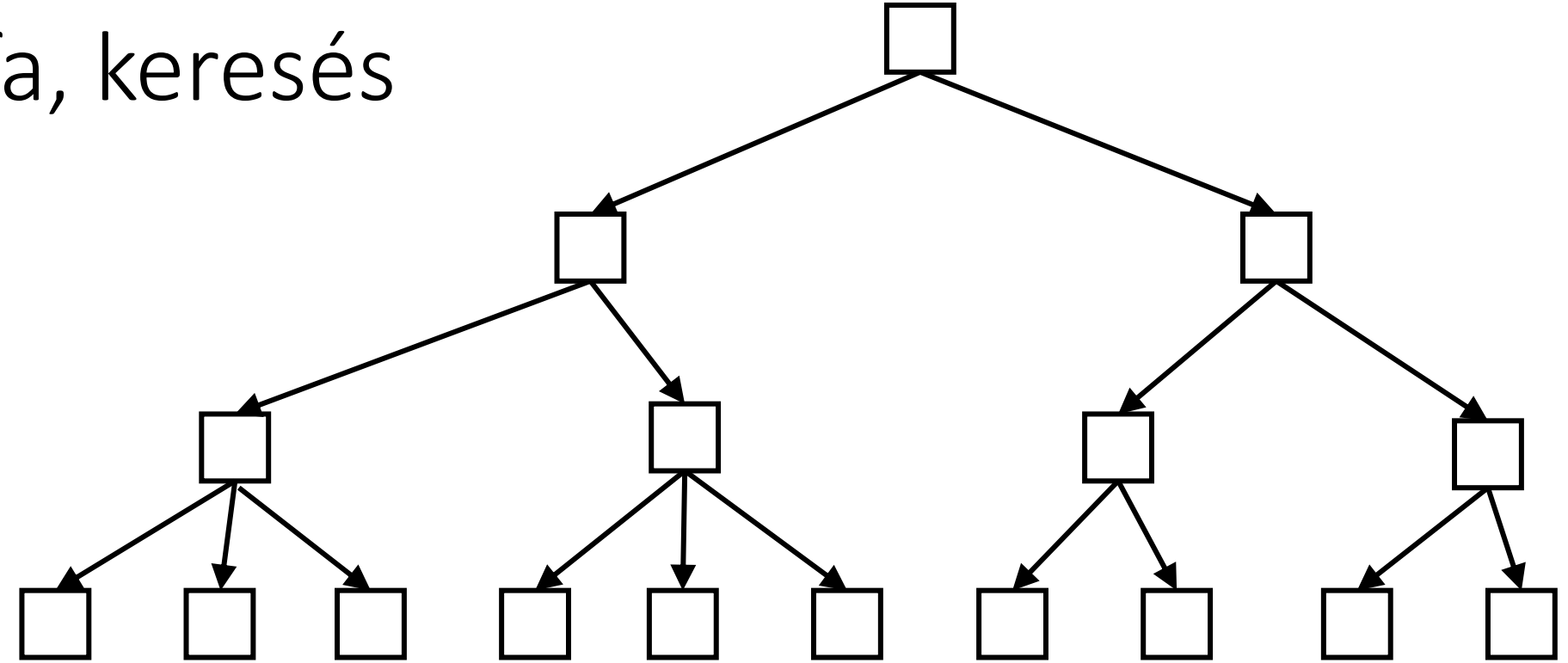
B*-fa, keresés

- 18 keresése
- Minden indexblokkolvasás után egyetlen mutatót követünk
  - A keresési tér f_i-ed részére csökken, tehát az index (és az adatállomány) f_i-ed részét vesszük figyelembe a következő keresési lépéstől kezdve
  - **Nem „keresünk”** a blokkok között, hanem mutatók mentén navigálunk
  - $\lceil \log_{f_i}(b_i + 1) \rceil + 1 = HT_i$



B*-fa, keresés

B*-fa

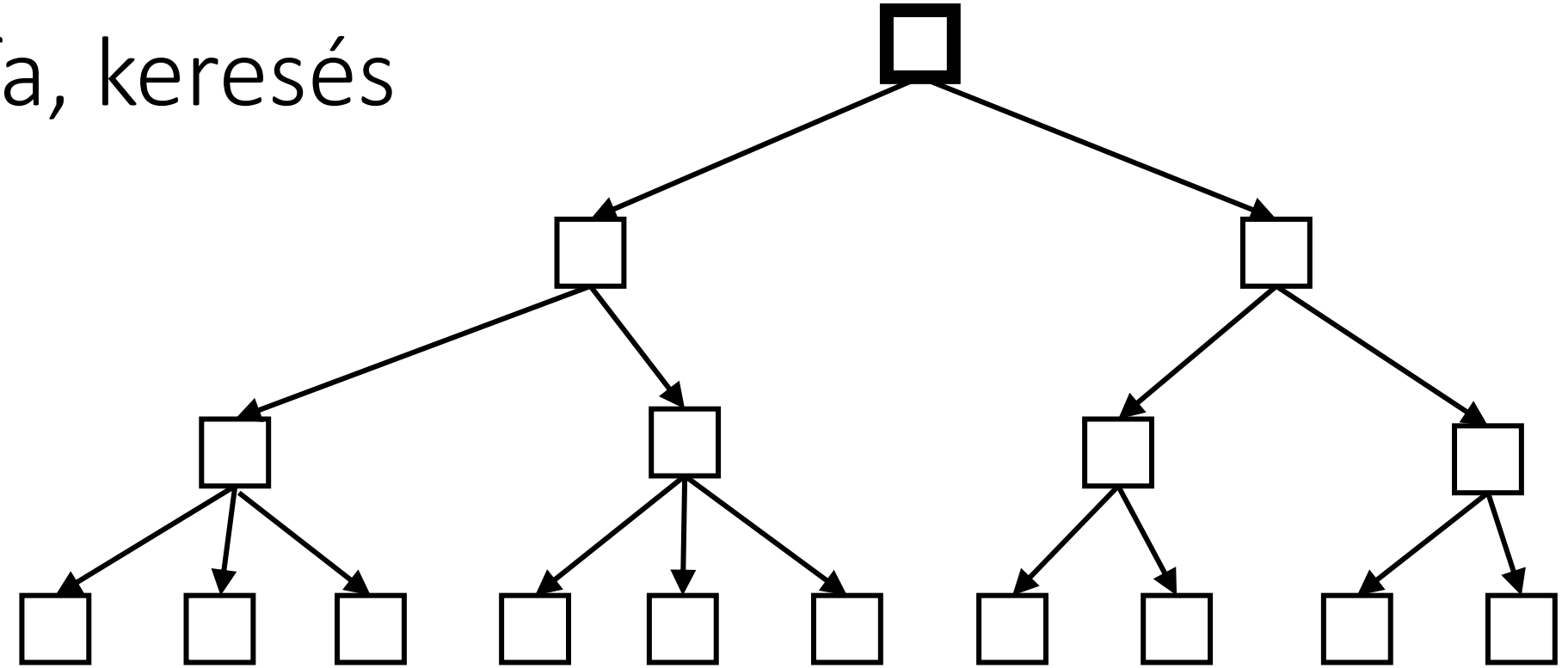


adatállomány



B*-fa, keresés

B*-fa

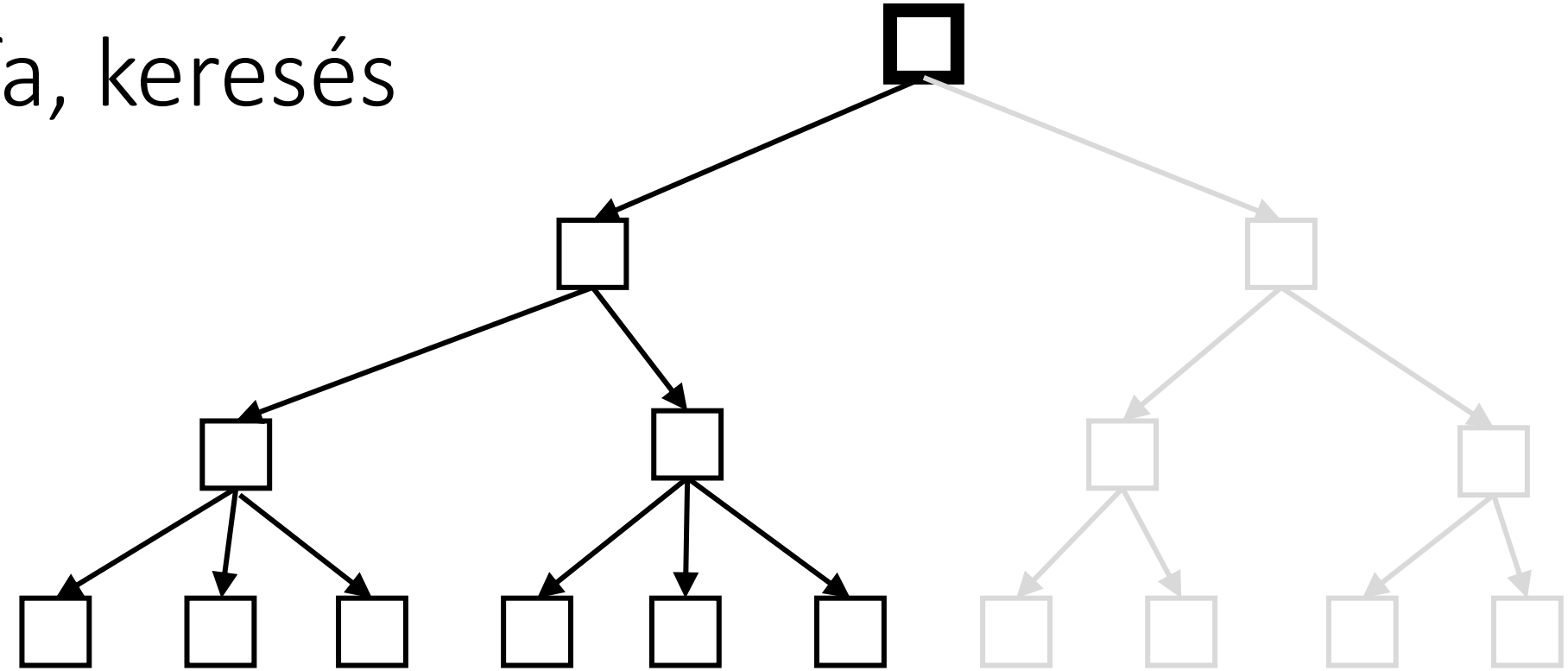


adatállomány



B*-fa, keresés

B*-fa

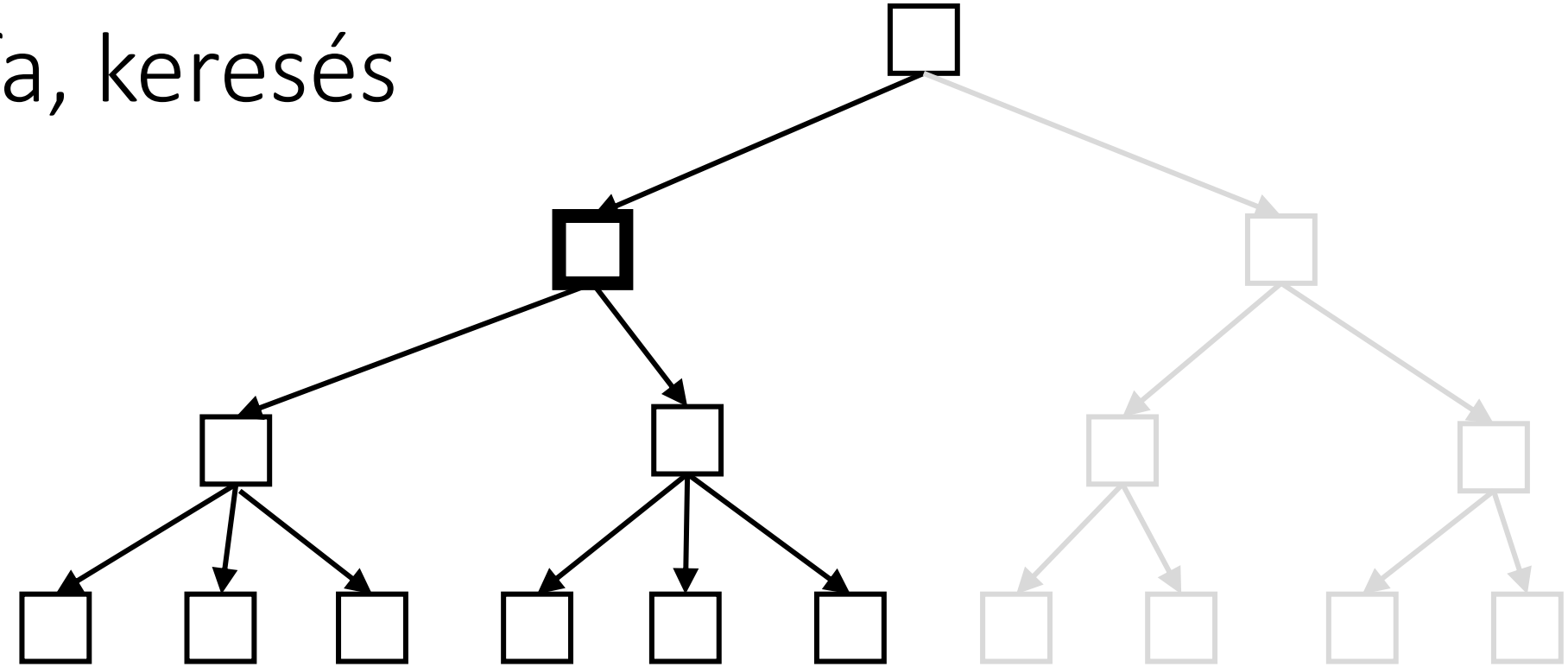


adatállomány



B*-fa, keresés

B*-fa

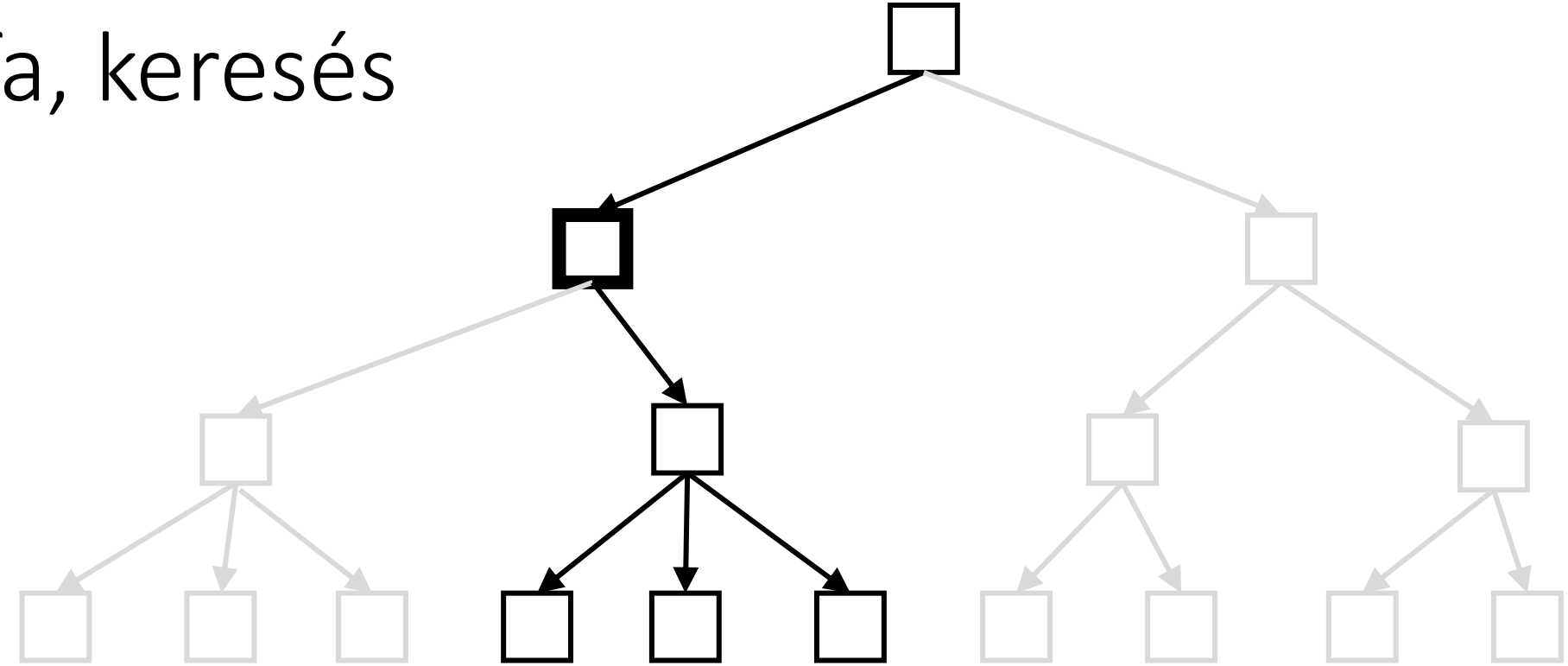


adatállomány



B*-fa, keresés

B*-fa

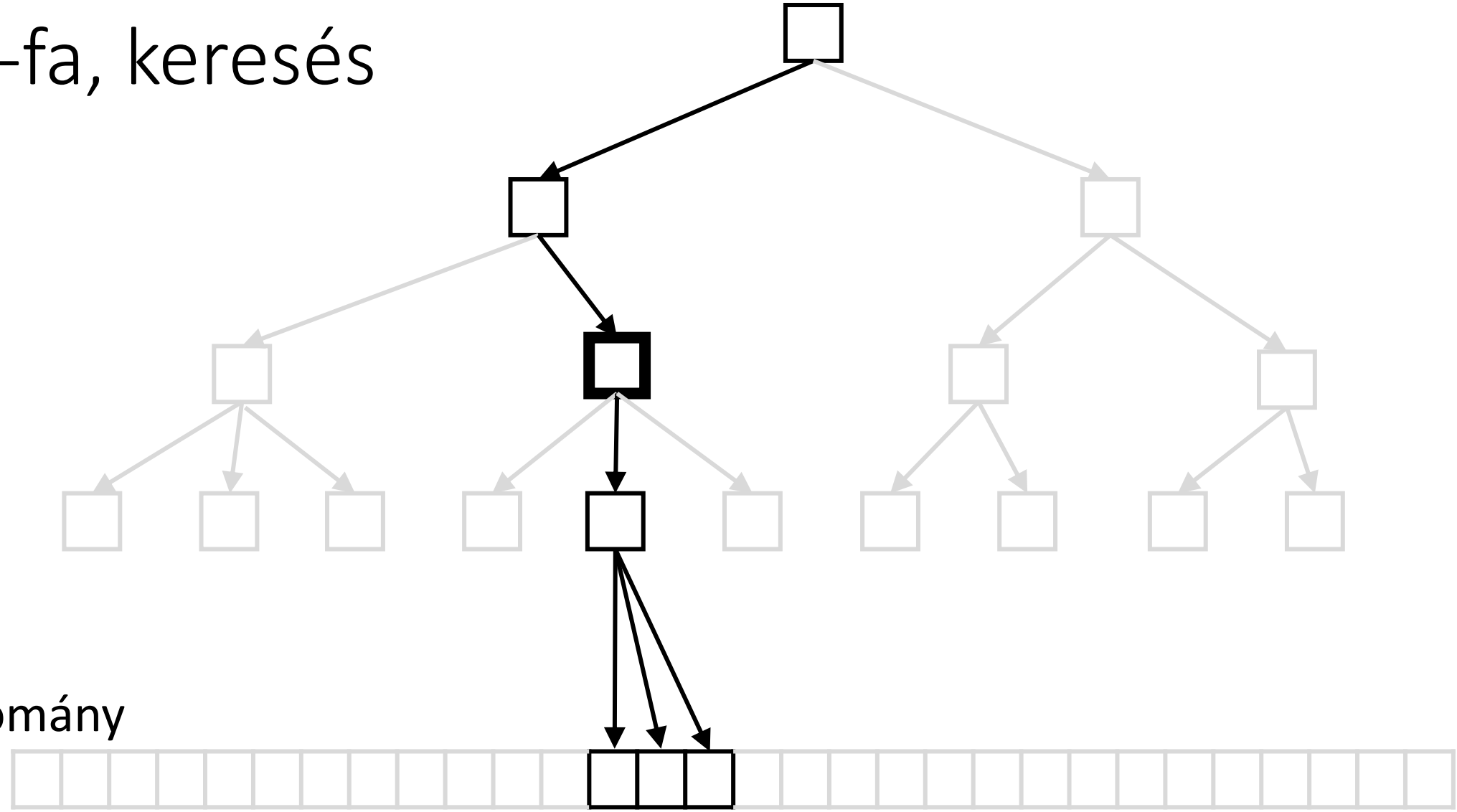


adatállomány



B*-fa, keresés

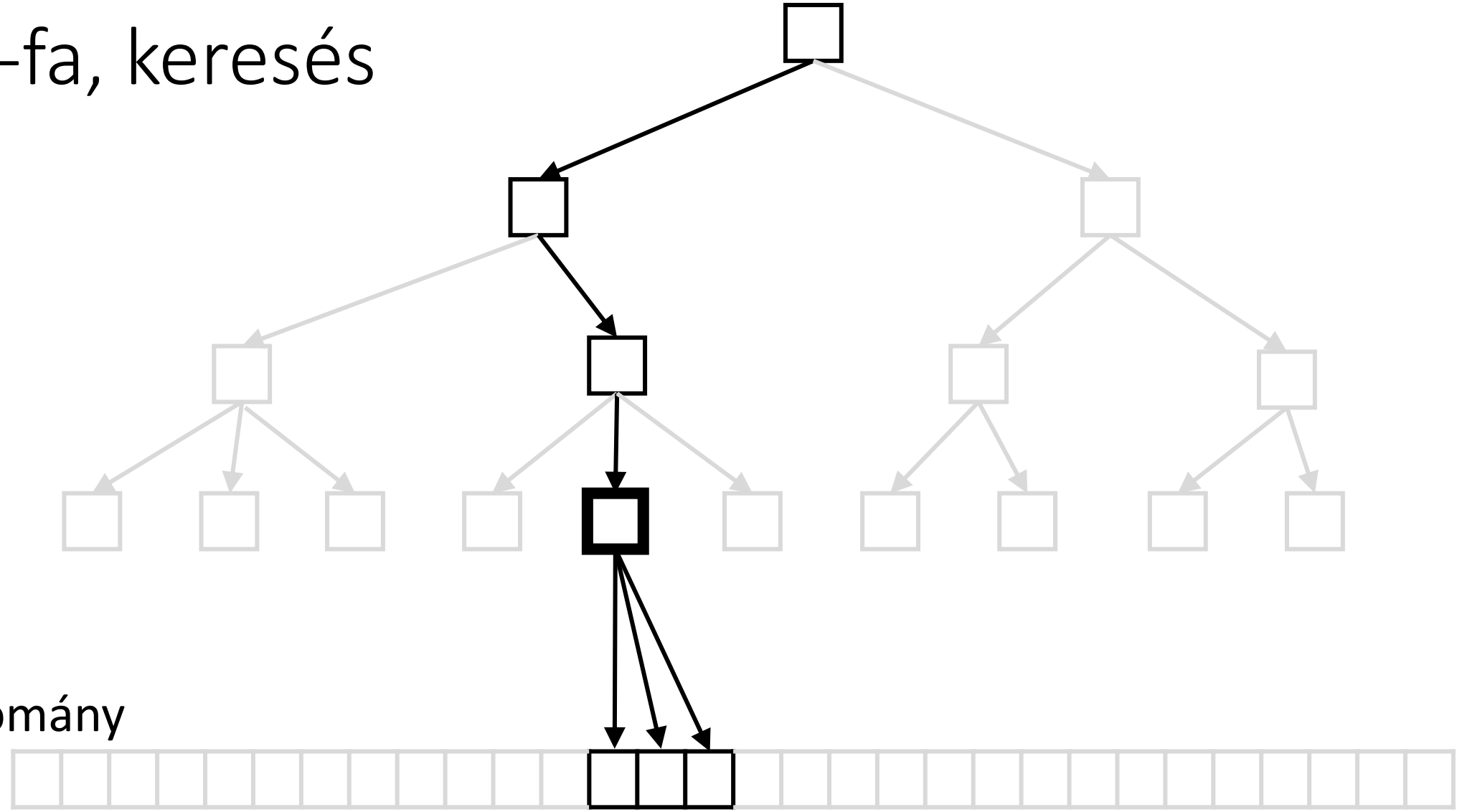
B*-fa



adatállomány

B*-fa, keresés

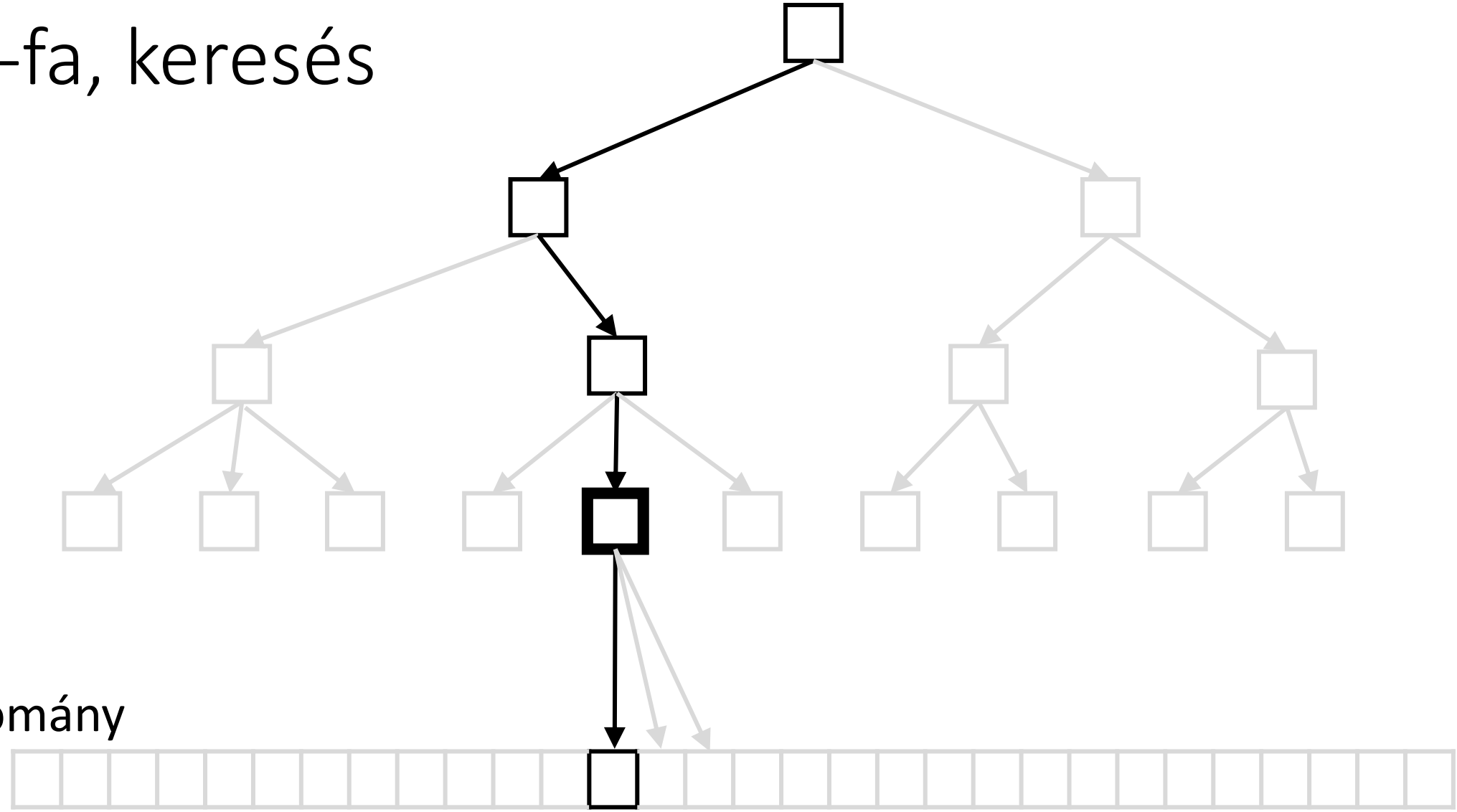
B*-fa



adatállomány

B*-fa, keresés

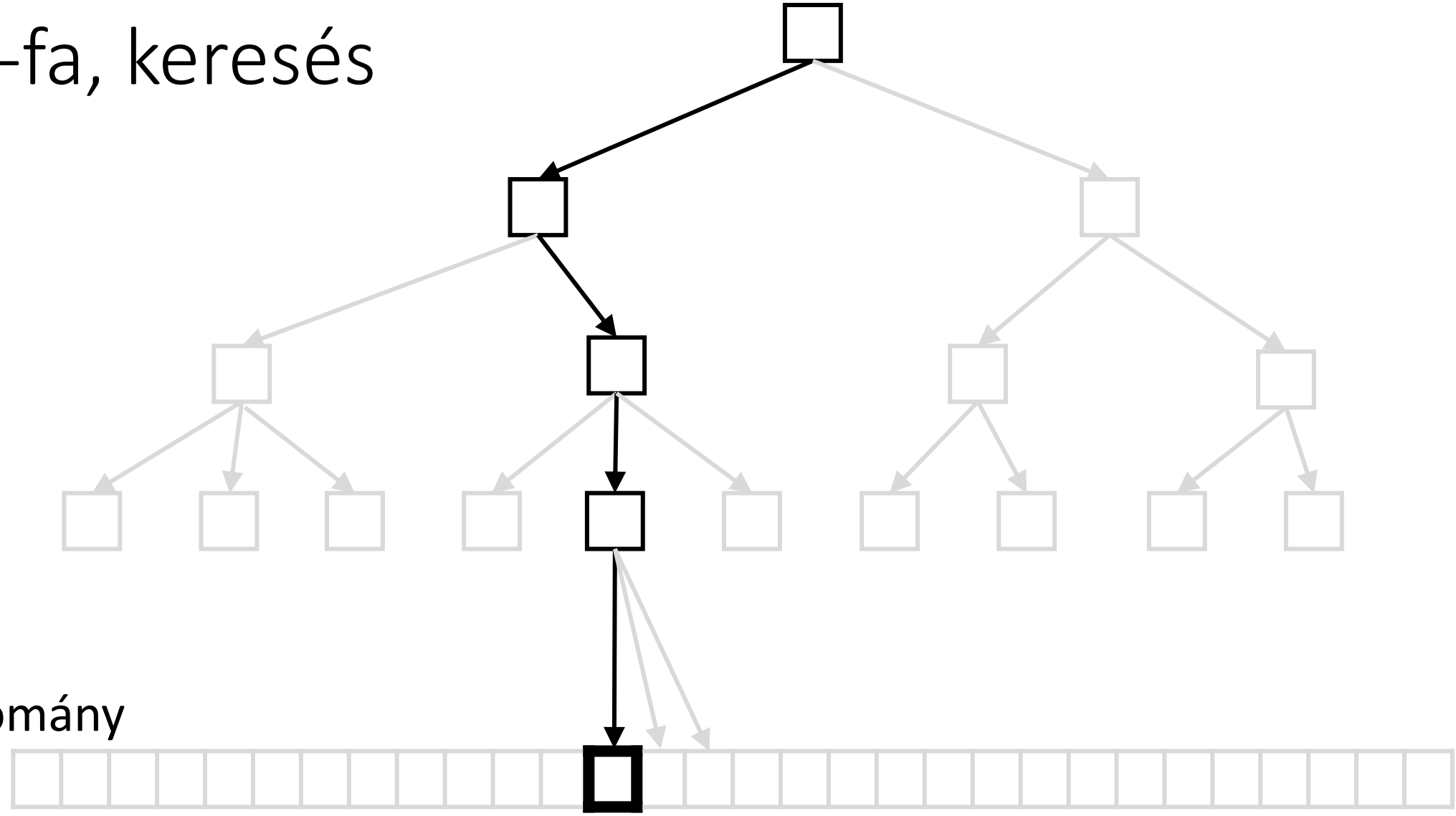
B*-fa



adatállomány

B*-fa, keresés

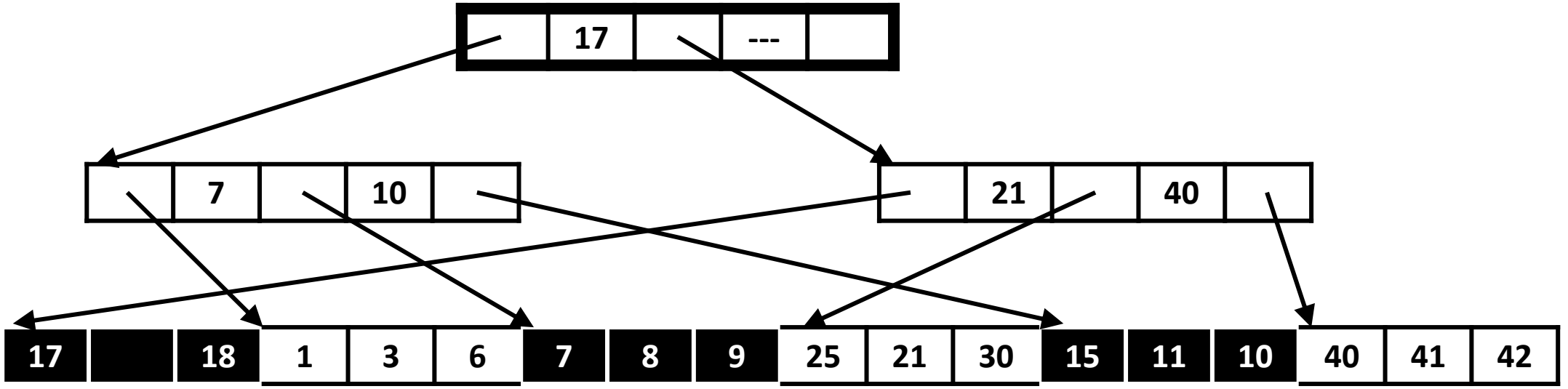
B*-fa



adatállomány

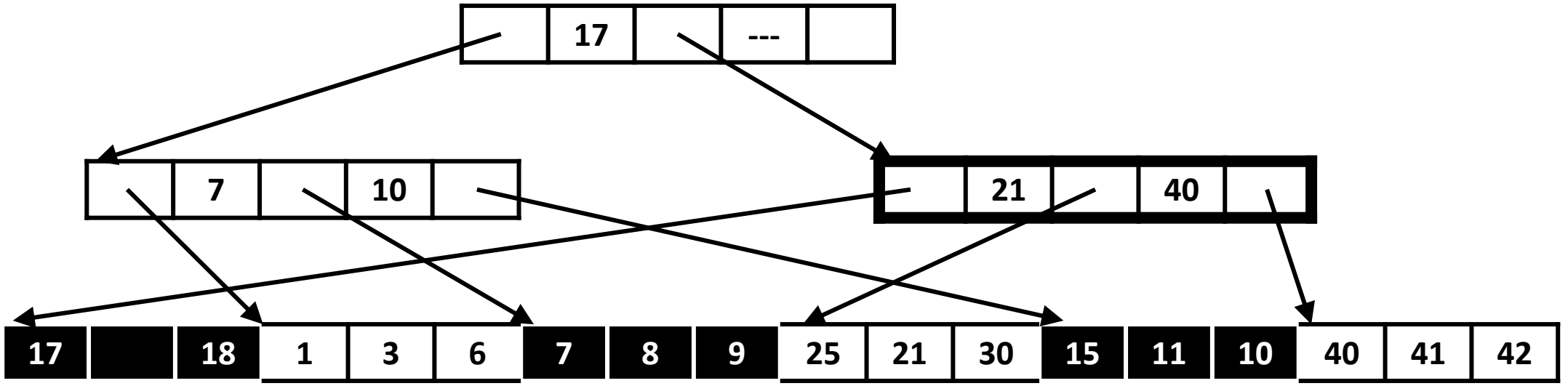
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 1. eset: 19 beillesztése



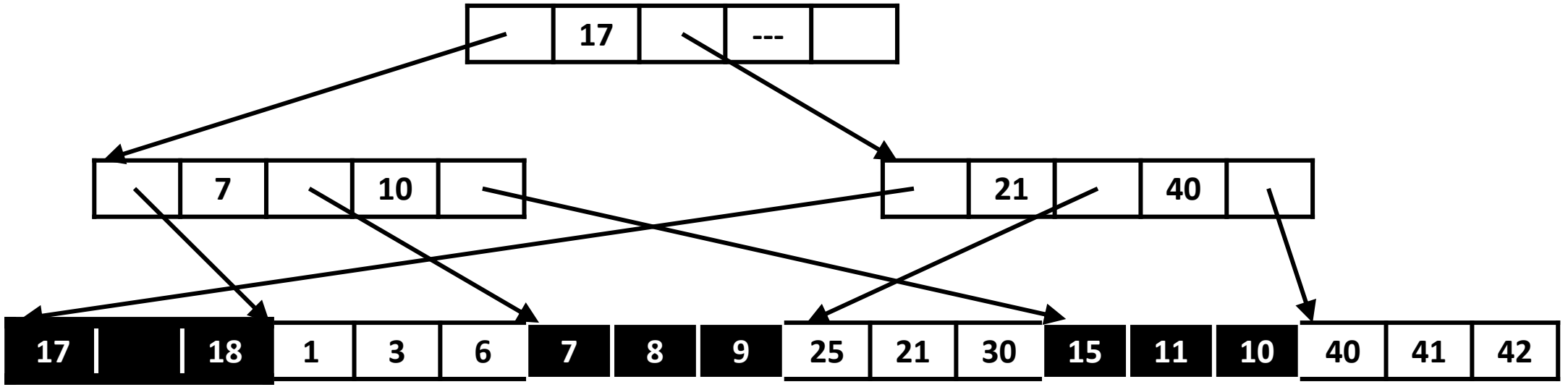
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 1. eset: 19 beillesztése



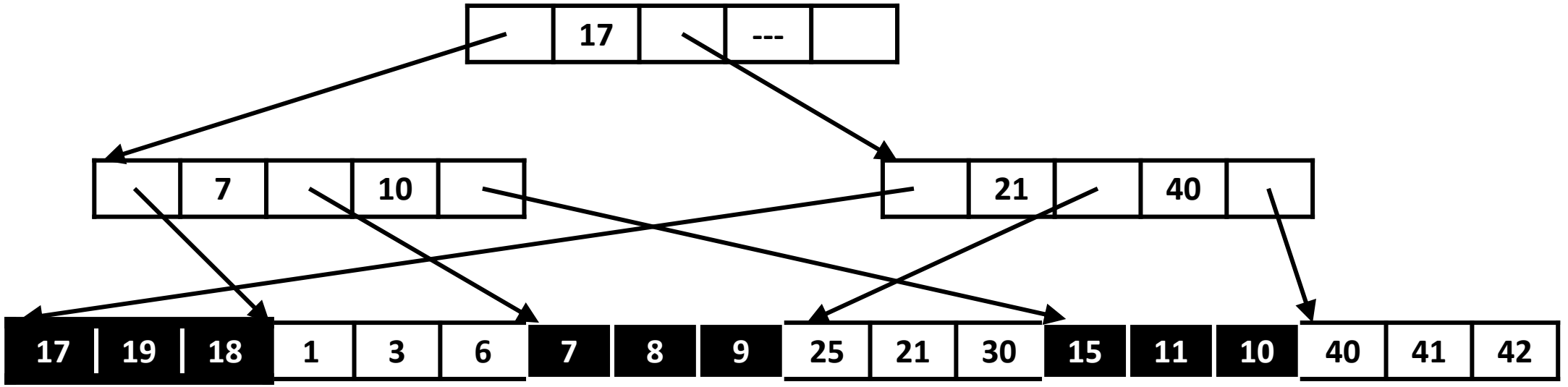
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 1. eset: 19 beillesztése



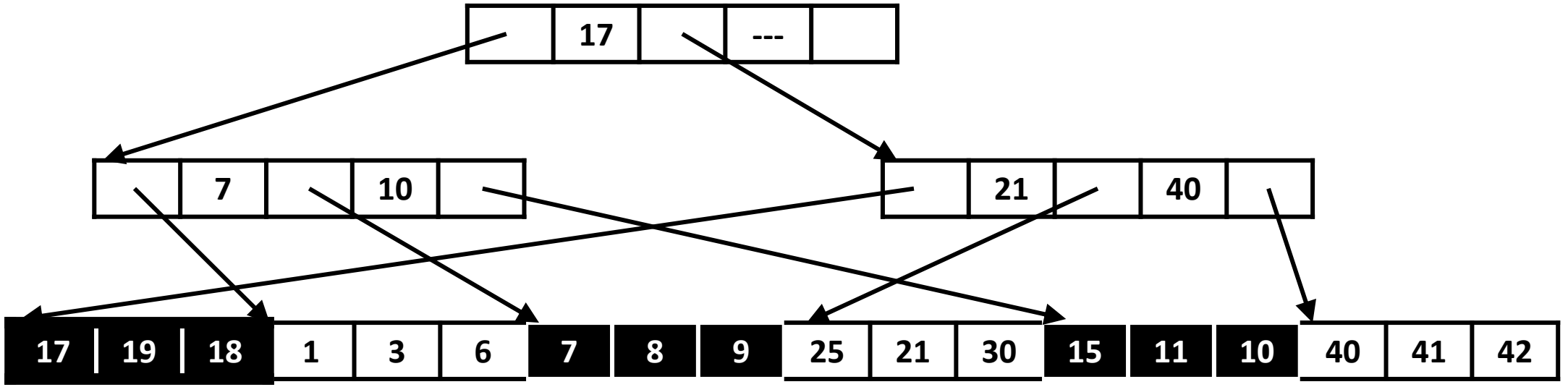
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 1. eset: 19 beillesztése



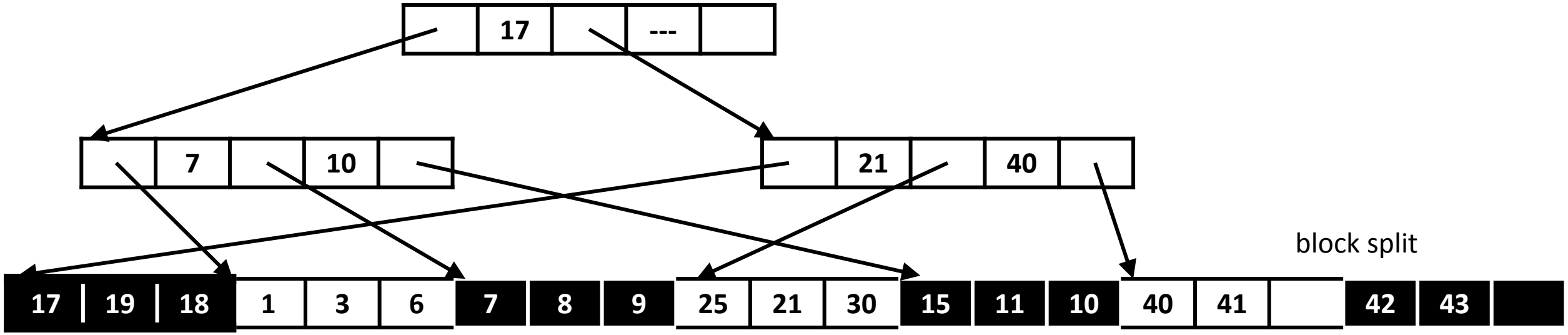
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 2. eset: 43 beillesztése – blokk megtelt, hasítani kell
  - Eredményezheti indexblokkok hasítását is
  - Eredményezheti a gyökér hasítását → új gyökér, +1 réteg (esetünkben nem)



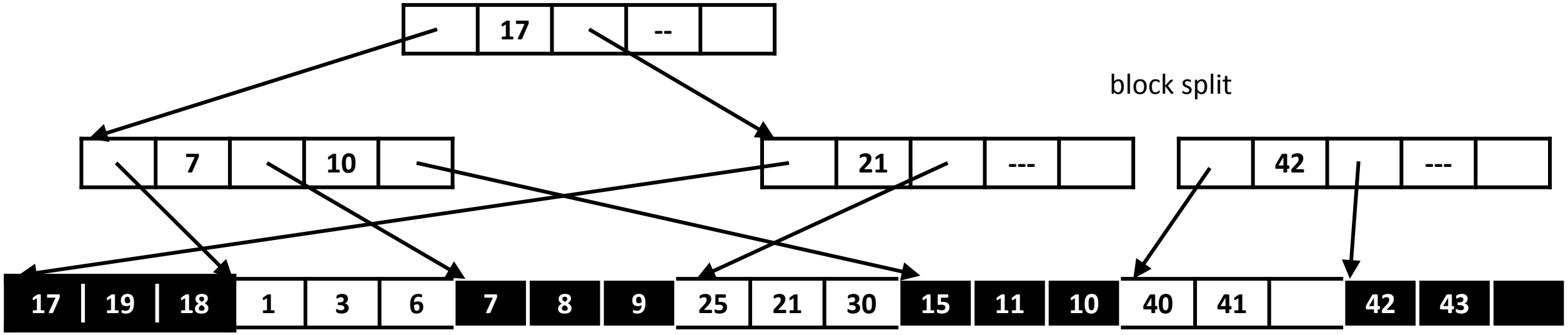
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 2. eset: 43 beillesztése – blokk megtelt, hasítani kell
  - Eredményezheti indexblokkok hasítását is
  - Eredményezheti a gyökér hasítását → új gyökér, +1 réteg (esetünkben nem)



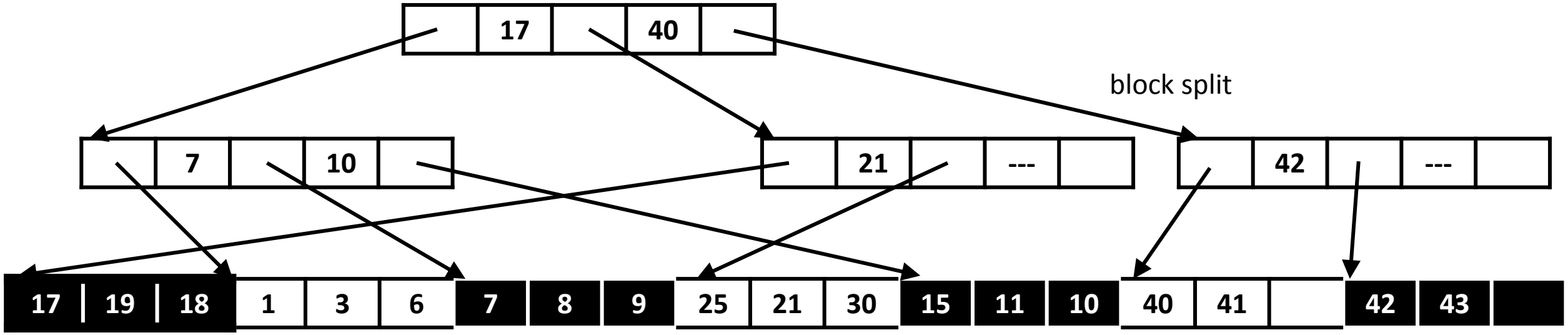
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 2. eset: 43 beillesztése – blokk megtelt, hasítani kell
  - Eredményezheti indexblokkok hasítását is
  - Eredményezheti a gyökér hasítását → új gyökér, +1 réteg (esetünkben nem)



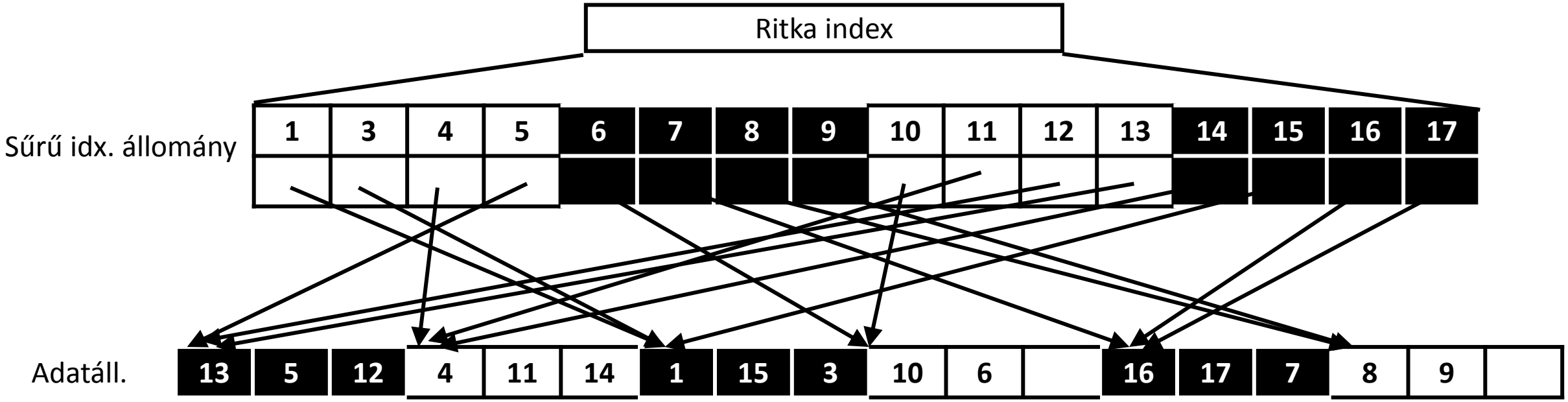
B*-fa, beillesztés

- Keresés+beillesztés a megfelelő blokkba
- 2. eset: 43 beillesztése – blokk megtelt, hasítani kell
  - Eredményezheti indexblokkok hasítását is
  - Eredményezheti a gyökér hasítását → új gyökér, +1 réteg (esetünkben nem)



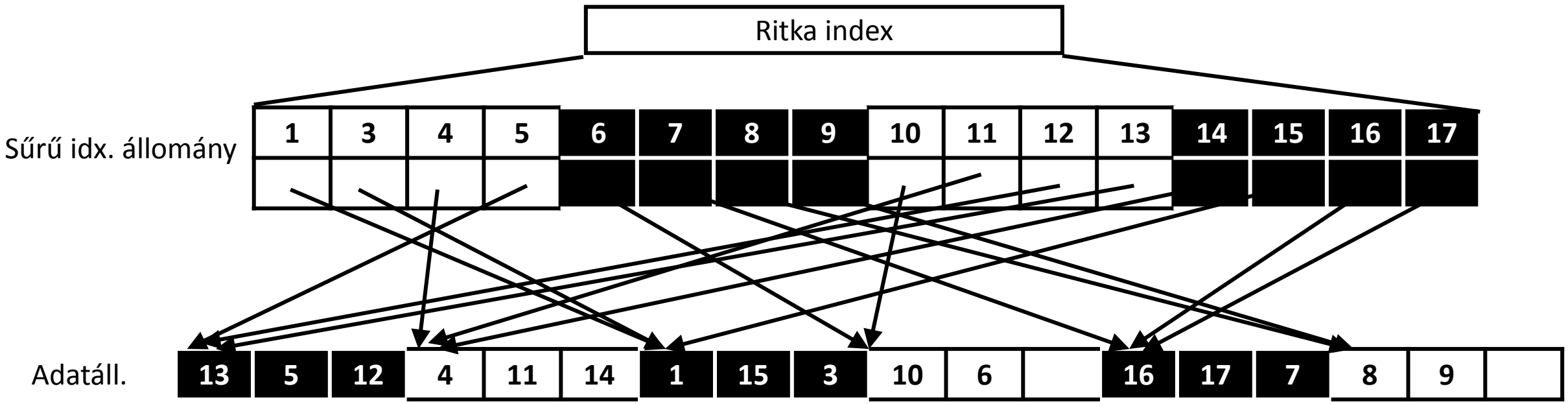
Sűrű index

- Kulcs+mutató (k, p) párok az indexállományban
- Az index kulcsértéke a mutatott adatrekord keresésikulcs-értéke
- A mutatók általában csak adatblokkot azonosítanak
- 1 indexrekord minden adat**rekord**hoz
- Adatállomány rendezetlen
- Nem önálló állományszervezés – ritka index/hash van a „tetején”



Sűrű index

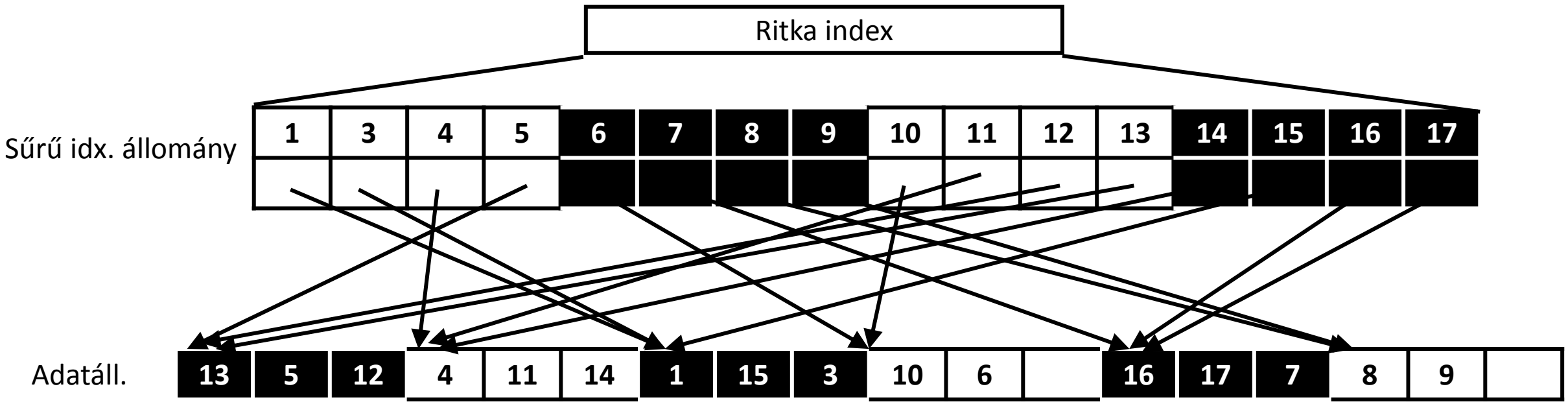
Indexállomány blocking faktora: $f_i = \left\lfloor \frac{b}{p+k} \right\rfloor$



Sűrű index, keresés

- Ritka index / B*-fa keresése
  +1 blokkolvasás a sűrű indexből
  +1 blokkolvasás az adatállományból

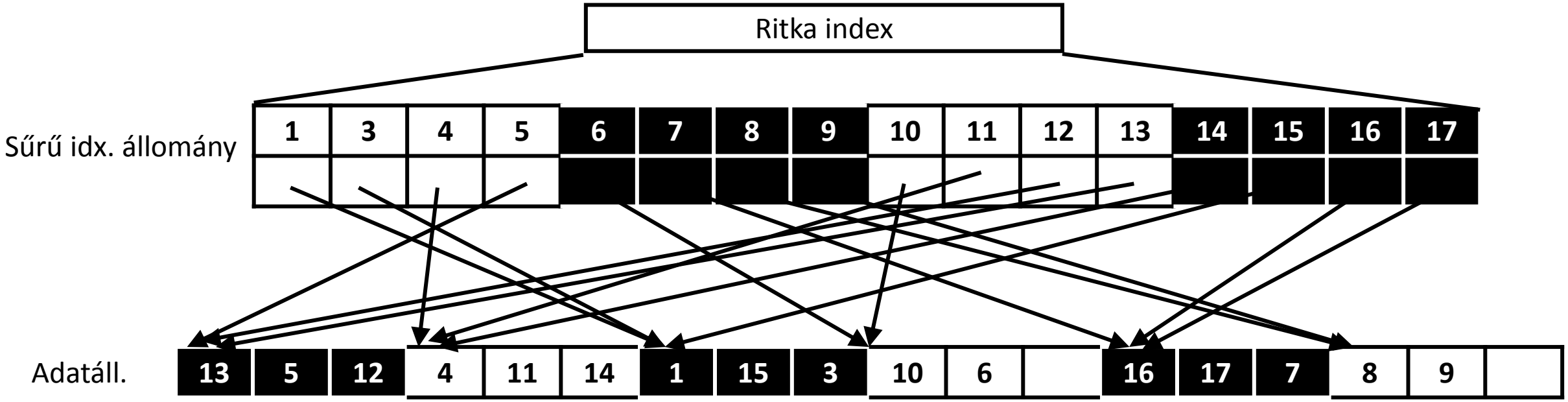
(Akkor ez mire jó? Ráadásul több tárhely, …



Sűrű index, beillesztés

- Beillesztés az adatállományba, akárhova
- Új indexbejegyzés létrehozása a rendezett sűrű index állományban
  (ha kell, blokkhasítással és a sűrű index/B*-fa módosításával)

… több karbantartás)



Sűrű index előnyei (?)

Sűrű index előnyei

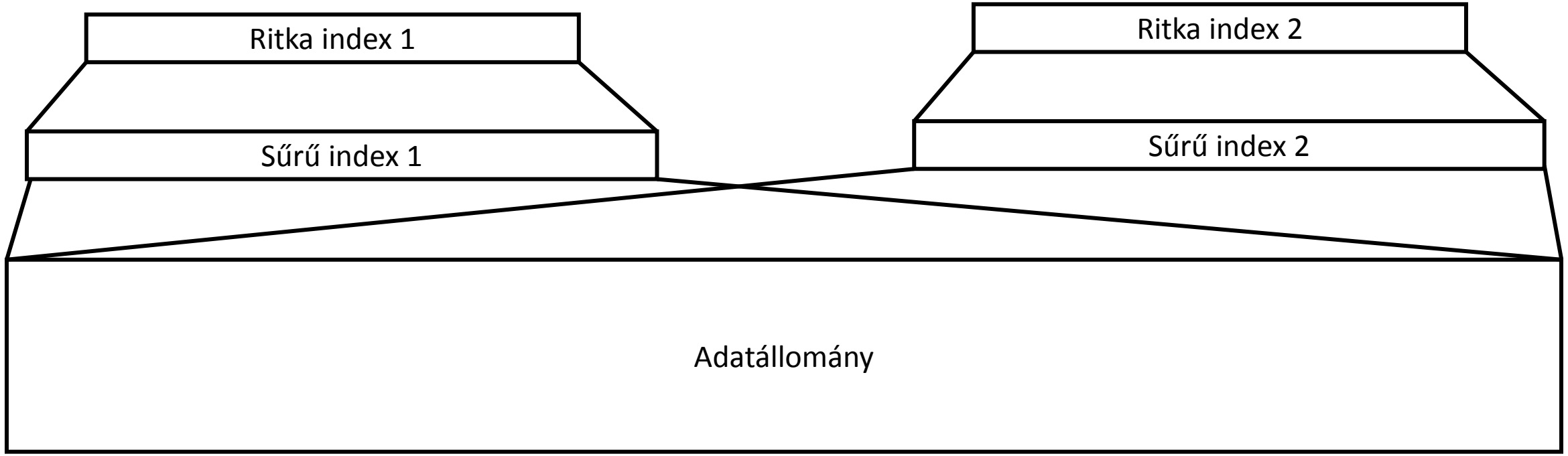
- Háttértár kihasználtsága javul (noha ez kevésbé fontos szempont)

Sűrű index előnyei

- Háttértár kihasználtsága javul (noha ez kevésbé fontos szempont)
- Több kulcs szerinti indexelt keresés támogatása
  - Pl.: keresés név VAGY telefonszám alapján

Sűrű index előnyei

- Háttértár kihasználtsága javul (noha ez kevésbé fontos szempont)
- Több kulcs szerinti indexelt keresés támogatása
  - Pl.: keresés név VAGY telefonszám alapján



Sűrű index előnyei

- Háttértár kihasználtsága javul (noha ez kevésbé fontos szempont)
- Több kulcs szerinti indexelt keresés támogatása
- Keresési idő (tipikus esetekben) csökken (Miért?)